

รายงาน

เรื่อง ระบบบริหารจัดการสมาชิกฟิตเนส Gym Member Management System

จัดทำโดย

นายฉัตรดนัย ไกรราช	รหัสนักศึกษา 673380037-1
นายภูริ ตั้งพงษ์	รหัสนักศึกษา 673380086-8
นายชินวัตร แสนเมือง	รหัสนักศึกษา 673380400-8
นายเกียรติพงษ์ เผติศักดิ์	รหัสนักศึกษา 673380433-3
นายศุภวัฒน์ แสนเรียน	รหัสนักศึกษา 673380604-2

เสนอ

รศ.ดร.ปัญญาพล หอระตะ

วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353002 หลักการออกแบบพัฒนาซอฟต์แวร์

Principles of Software Design and Development

วิทยาลัยการคอมพิวเตอร์ สาขาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

บทคัดย่อ

โครงการ Gym Member Management System หรือระบบจัดการสมาชิกฟิตเนส มีวัตถุประสงค์เพื่อพัฒนาระบบสำหรับจัดเก็บและจัดการข้อมูลที่เกี่ยวข้องกับการให้บริการฟิตเนสอย่างเป็นระบบ ลดความซ้ำซ้อนในการจัดการข้อมูล และเพิ่มความสะดวกในการเข้าถึงข้อมูล โดยระบบครอบคลุมการจัดการข้อมูลเทรนเนอร์ สมาชิก ข้อมูลเพิ่มเติมของสมาชิก แผนสมาชิก การสมัครสมาชิก และตารางฝึกซ้อม

ระบบพัฒนาด้วยภาษา Java 17 และ Spring Boot ใช้ PostgreSQL เป็นระบบจัดการฐานข้อมูล และใช้ Spring Data JPA สำหรับการเชื่อมต่อและจัดการข้อมูล ใช้ Thymeleaf ในการพัฒนาส่วนติดต่อผู้ใช้ และใช้ Maven ในการจัดการโครงการ โดยออกแบบสถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture) ร่วมกับรูปแบบ Model-View-Controller (MVC) เพื่อแยกหน้าที่การทำงานของแต่ละส่วนให้ชัดเจน

นอกจากนี้ยังประยุกต์ใช้ Strategy Pattern สำหรับการคำนวณราคาตามประเภทลูกค้า และ State Pattern สำหรับการจัดการสถานะของตารางฝึกซ้อม รวมถึงใช้ Docker เพื่อจัดเตรียมสภาพแวดล้อมสำหรับการเรียกใช้งานระบบ

ผลการพัฒนาแสดงให้เห็นว่าระบบสามารถจัดการข้อมูลหลักผ่าน REST API และหน้าเว็บได้ตามขอบเขตที่กำหนด โดยมีการทดสอบระดับหน่วย (Unit Testing) ด้วย JUnit 5 และ Mockito ครอบคลุมคลาสทดสอบ 6 คลาส รวมกรณีทดสอบที่ผ่านทั้งหมด 59 กรณี นอกจากนี้ ระบบยังมีการจัดเตรียมสภาพแวดล้อมด้วย Docker เพื่อสนับสนุนการติดตั้งและเรียกใช้งาน ผลจากการดำเนินโครงการทำให้ได้ระบบที่มีโครงสร้างชัดเจน แบ่งหน้าที่การทำงานเป็นสัดส่วน และสามารถนำไปบำรุงรักษาหรือต่อยอดในอนาคตได้

คำสำคัญ ระบบจัดการสมาชิกฟิตเนส, Spring Boot, PostgreSQL, REST API, Software Design Pattern

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design and Development (หลักการออกแบบพัฒนาซอฟต์แวร์) จัดทำขึ้นเพื่อศึกษาและประยุกต์ใช้หลักการออกแบบและพัฒนาซอฟต์แวร์ผ่านโครงงาน Gym Member Management System (ระบบจัดการสมาชิกฟิตเนส) ซึ่งมีวัตถุประสงค์เพื่อพัฒนาระบบสำหรับจัดการข้อมูลสมาชิก เทรนเนอร์ แผนสมาชิก การสมัครสมาชิก และตารางฝึกซ้อมให้เป็นระบบ

เนื้อหาในรายงานประกอบด้วยการวิเคราะห์ความต้องการของระบบ การออกแบบสถาปัตยกรรมซอฟต์แวร์ การออกแบบฐานข้อมูล การพัฒนาฟังก์ชันต่าง ๆ การประยุกต์ใช้รูปแบบการออกแบบซอฟต์แวร์ (Software Design Patterns) ตลอดจนการทดสอบและประเมินผลการทำงานของระบบ โดยใช้เทคโนโลยี Java 17, Spring Boot, PostgreSQL และ Thymeleaf ในการพัฒนา เพื่อให้ระบบมีโครงสร้างที่ชัดเจน สามารถดูแลรักษาและพัฒนาต่อยอดได้

การจัดทำโครงงานนี้มุ่งเน้นการนำความรู้จากรายวิชามาใช้ในการแก้ปัญหาอย่างเป็นขั้นตอน ตั้งแต่การวางแผนและออกแบบระบบ ไปจนถึงการพัฒนาและทดสอบโปรแกรม รวมถึงการทำงานร่วมกันเป็นทีม และการจัดการซอร์สโค้ดด้วย Git และ GitHub ซึ่งช่วยให้ผู้จัดทำได้เรียนรู้กระบวนการพัฒนาซอฟต์แวร์และเข้าใจการทำงานของระบบในภาพรวมมากยิ่งขึ้น

ผู้จัดทำหวังว่ารายงานฉบับนี้จะช่วยให้ผู้อ่านเข้าใจแนวทางการออกแบบและพัฒนาระบบจัดการสมาชิกฟิตเนส ตลอดจนการประยุกต์ใช้หลักการออกแบบซอฟต์แวร์กับการพัฒนาโปรแกรมจริง หากเนื้อหาในรายงานมีข้อบกพร่องประการใด ผู้จัดทำยินดีรับข้อเสนอแนะเพื่อนำไปปรับปรุงให้มีความถูกต้องและสมบูรณ์ยิ่งขึ้นต่อไป

คณะผู้จัดทำ

วันที่ 9 ตุลาคม 2569

สารบัญ

	หน้า
บทคัดย่อ	ก
คำนำ	ข
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญของโครงการ	
1.2 วัตถุประสงค์ของโครงการ, 1.3 ขอบเขตของโครงการ	2
1.4 กลุ่มผู้ใช้งานระบบ, 1.5 ประโยชน์ที่คาดว่าจะได้รับ	4
บทที่ 2 แนวคิด ทฤษฎี และเทคโนโลยีที่เกี่ยวข้อง	6
2.1 แนวคิดเกี่ยวกับระบบจัดการสมาชิกฟิตเนส	
2.2 แนวคิดเกี่ยวกับการพัฒนาเว็บแอปพลิเคชัน	
2.3 สถาปัตยกรรมแบบชั้น (Layered Architecture)	7
2.4 รูปแบบ Model-View-Controller (MVC)	8
2.5 หลักการออกแบบซอฟต์แวร์ SOLID	9
2.6 รูปแบบการออกแบบซอฟต์แวร์ (Design Patterns)	10
2.7 แนวคิดเกี่ยวกับฐานข้อมูลเชิงสัมพันธ์	11
2.8 เทคโนโลยีที่ใช้ในการพัฒนาระบบ	12
2.9 แนวคิดเกี่ยวกับ REST API	15
2.10 แนวคิดเกี่ยวกับการทดสอบซอฟต์แวร์	16
2.11 แนวคิดเกี่ยวกับ Containerization และการนำระบบไปใช้งาน	17
2.12 สรุปแนวคิดและเทคโนโลยีที่เกี่ยวข้อง	18
บทที่ 3 การวิเคราะห์และออกแบบระบบ	19
3.1 การวิเคราะห์ปัญหาและความต้องการของระบบ	
3.2 การกำหนดความต้องการของระบบ	20
3.3 การออกแบบสถาปัตยกรรมระบบ	22
3.4 การออกแบบฐานข้อมูล	24
3.5 การออกแบบกฎทางธุรกิจ	28
3.6 การออกแบบ REST API	29
3.7 การออกแบบส่วนติดต่อผู้ใช้	31

3.8 การออกแบบการทดสอบระบบ	
3.9 การออกแบบการทำงานร่วมกันของทีม	
3.10 สรุปผลการวิเคราะห์และออกแบบระบบ	32
บทที่ 4 การพัฒนาและผลการดำเนินงาน	33
4.1 ภาพรวมผลการพัฒนา	
4.2 ผลการพัฒนาระบบจัดการข้อมูล	
4.3 ผลการพัฒนา REST API	34
4.4 ผลการพัฒนาส่วนติดต่อผู้ใช้	
4.5 ผลการประยุกต์ใช้ Design Patterns	35
4.6 ผลการทดสอบซอฟต์แวร์	36
4.7 ผลการจัดเตรียมระบบด้วย Docker	37
4.8 ปัญหาที่พบระหว่างการพัฒนาและแนวทางแก้ไข	38
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ	40
5.1 สรุปผลการพัฒนาระบบ	
5.2 อภิปรายผลการพัฒนา	
5.3 ข้อจำกัดของระบบ	41
5.4 ข้อเสนอแนะในการพัฒนาต่อ	
5.5 บทสรุป	42
เอกสารอ้างอิง	43

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของโครงการ

ปัจจุบันการให้บริการพัฒนาระบบข้อมูลที่เกี่ยวข้องกับการดำเนินงานหลายประเภท เช่น ข้อมูลสมาชิก ข้อมูลเทรดเดอร์ ข้อมูลส่วนตัวของสมาชิก แพ็กเกจสมาชิก ประวัติการสมัครสมาชิก และการนัดหมายฝึกซ้อม ข้อมูลเหล่านี้มีความสัมพันธ์กันและจำเป็นต้องได้รับการจัดเก็บอย่างเป็นระบบ เพื่อให้พนักงานหรือผู้ดูแลฟิตเนส สามารถตรวจสอบข้อมูลและดำเนินงานได้อย่างถูกต้องและมีประสิทธิภาพ

การจัดเก็บข้อมูลด้วยวิธีการทั่วไป เช่น การจดบันทึกลงในสมุดหรือการใช้โปรแกรมตารางคำนวณ อาจทำให้ข้อมูลกระจัดกระจาย ค้นหาข้อมูลได้ไม่สะดวก และเกิดข้อมูลซ้ำซ้อนได้ นอกจากนี้ การคำนวณวันหมดอายุของสมาชิกด้วยตนเองยังมีโอกาสเกิดข้อผิดพลาด ขณะที่การนัดหมายฝึกซ้อมกับเทรดเดอร์ จำเป็นต้องมีการบันทึกวัน เวลา และสถานะของการนัดหมาย เพื่อให้สามารถตรวจสอบได้ว่าการนัดหมายได้รับการจอง ดำเนินการเสร็จสิ้น หรือถูกยกเลิกแล้ว อีกทั้งการกำหนดราคาสำหรับลูกค้าแต่ละประเภท เช่น ลูกค้าทั่วไป นักศึกษา และลูกค้าประจำ ยังจำเป็นต้องใช้หลักเกณฑ์การคำนวณที่ชัดเจนและสอดคล้องกัน

จากปัญหาดังกล่าว จึงเกิดแนวคิดในการพัฒนาระบบจัดการสมาชิกฟิตเนส (Gym Member Management System) ในรูปแบบเว็บแอปพลิเคชัน เพื่อรวบรวมข้อมูลที่เกี่ยวข้องกับการดำเนินงานของฟิตเนสไว้ในระบบเดียว โดยพัฒนาความสามารถในการเพิ่ม ดู แก้ไข และลบข้อมูล ตลอดจนกำหนดให้ระบบคำนวณวันหมดอายุของสมาชิกโดยอัตโนมัติ รองรับการจัดการนัดหมายฝึกซ้อม และคำนวณราคาตามประเภทลูกค้า ระบบยังมีส่วนติดต่อผู้ใช้ผ่านเว็บและ REST API เพื่อให้ผู้ใช้งานสามารถจัดการข้อมูลผ่านหน้าเว็บ ขณะที่ระบบหรือโปรแกรมอื่นสามารถเรียกใช้ข้อมูลผ่าน API ได้

ในการพัฒนาโครงการ ทีมงานประยุกต์ใช้ภาษา Java และ Spring Boot ร่วมกับฐานข้อมูล PostgreSQL โดยออกแบบโครงสร้างซอฟต์แวร์ตามสถาปัตยกรรมแบบชั้น (Layered Architecture) และรูปแบบ Model-View-Controller (MVC) เพื่อแยกส่วนรับคำขอ ตรรกะทางธุรกิจ การเข้าถึงข้อมูล และส่วนแสดงผลออกจากกัน นอกจากนี้ ยังนำหลักการ SOLID และรูปแบบการออกแบบซอฟต์แวร์ (Design Patterns) มาใช้ในการจัดโครงสร้างโปรแกรม โดยเฉพาะรูปแบบ Strategy สำหรับการคำนวณราคา รูปแบบ State สำหรับการจัดการสถานะนัดหมาย และแนวคิด Observer สำหรับการแจ้งเหตุการณ์เมื่อสถานะนัดหมายเปลี่ยนแปลง

โครงการนี้เป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design and Development โดยมุ่งเน้นทั้งการพัฒนาระบบให้สามารถทำงานตามขอบเขตที่กำหนด และการประยุกต์ใช้หลักการออกแบบซอฟต์แวร์ในกระบวนการพัฒนาจริง รวมถึงการทำงานร่วมกันผ่าน Git และ GitHub การทดสอบซอฟต์แวร์ด้วย Unit Test และการจัดเตรียมระบบให้ทำงานผ่าน Docker ซึ่งช่วยให้สมาชิกในทีมสามารถแบ่งความรับผิดชอบและพัฒนาส่วนต่าง ๆ ของระบบภายใต้แนวทางการออกแบบเดียวกันได้

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อพัฒนาระบบจัดการข้อมูลของฟิตเนสให้สามารถเพิ่ม ดู แก้ไข และลบข้อมูลหลักทั้ง 6 ส่วน ได้แก่ ข้อมูลเทรนเนอร์ ข้อมูลสมาชิก ข้อมูลส่วนตัวของสมาชิก แพ็กเกจสมาชิก การสมัครสมาชิก และการนัดหมายฝึกซ้อม
2. เพื่อพัฒนาระบบให้สามารถให้บริการข้อมูลผ่าน REST API พร้อมเอกสารประกอบ API ที่สร้างด้วย Swagger UI และมีหน้าเว็บสำหรับจัดการข้อมูลผ่าน Thymeleaf
3. เพื่อประยุกต์ใช้สถาปัตยกรรมแบบชั้น หลักการ SOLID และรูปแบบการออกแบบซอฟต์แวร์ เพื่อให้ระบบมีการแบ่งหน้าที่อย่างชัดเจน ลดการทำงานซ้ำซ้อน และสะดวกต่อการบำรุงรักษา
4. เพื่อพัฒนาความสามารถในการคำนวณวันหมดอายุของสมาชิกโดยอัตโนมัติ การจัดการสถานะนัดหมายฝึกซ้อม และการคำนวณราคาแพ็กเกจตามประเภทลูกค้า
5. เพื่อฝึกกระบวนการพัฒนาซอฟต์แวร์ร่วมกันเป็นทีม โดยใช้ Git และ GitHub ในการควบคุมเวอร์ชันและตรวจสอบการเปลี่ยนแปลงของโค้ด ตลอดจนใช้ JUnit 5 และ Mockito ในการทดสอบซอฟต์แวร์
6. เพื่อพัฒนาและจัดเตรียมระบบด้วย Docker รวมถึงนำระบบไปติดตั้งและให้บริการบนแพลตฟอร์ม Cloud ผ่าน Render

1.3 ขอบเขตของโครงการ

โครงการนี้ครอบคลุมการพัฒนาระบบจัดการข้อมูลฟิตเนสผ่านเว็บแอปพลิเคชันและ REST API โดยแบ่งการทำงานออกเป็นส่วนต่าง ๆ ดังนี้

1.3.1 การจัดการข้อมูลเทรนเนอร์ ระบบสามารถเพิ่ม แสดงรายละเอียด แก้ไข และลบข้อมูลเทรนเนอร์ได้ โดยข้อมูลประกอบด้วยชื่อ เบอร์โทรศัพท์ และความเชี่ยวชาญของเทรนเนอร์ ข้อมูลดังกล่าวสามารถนำไปใช้เชื่อมโยงกับสมาชิกที่อยู่ภายใต้การดูแลและการนัดหมายฝึกซ้อมได้

1.3.2 การจัดการข้อมูลสมาชิก ระบบสามารถเพิ่ม แสดงรายละเอียด แก้ไข และลบข้อมูลบัญชีสมาชิก โดยจัดเก็บชื่อผู้ใช้ อีเมล รหัสผ่าน เบอร์โทรศัพท์ และข้อมูลเทรนเนอร์ประจำตัวตามที่กำหนดไว้ในระบบ ทั้งนี้ การส่งข้อมูลสมาชิกกลับผ่าน REST API จะไม่รวมฟิลด์รหัสผ่านไว้ใน Response เพื่อจำกัดการเปิดเผยข้อมูลดังกล่าวผ่านการตอบกลับของ API

1.3.3 การจัดการข้อมูลส่วนตัวของสมาชิก ระบบสามารถจัดเก็บและจัดการข้อมูลส่วนตัวของสมาชิก ได้แก่ ชื่อและนามสกุล เพศ วันเกิด และข้อมูลผู้ติดต่อกรณีฉุกเฉิน โดยกำหนดความสัมพันธ์ระหว่างสมาชิกกับข้อมูลส่วนตัวเป็นแบบหนึ่งต่อหนึ่ง (One-to-One) สมาชิกหนึ่งคนมีข้อมูลส่วนตัวได้ไม่เกินหนึ่งชุด

1.3.4 การจัดการแพ็กเกจสมาชิก ระบบสามารถเพิ่ม แสดงรายละเอียด แก้ไข และลบข้อมูลแพ็กเกจสมาชิก โดยข้อมูลประกอบด้วยชื่อแพ็กเกจ ราคา และจำนวนวันที่แพ็กเกจมีผลใช้งาน ข้อมูลนี้ใช้เป็นพื้นฐานสำหรับการสมัครสมาชิกและการคำนวณวันหมดอายุ

1.3.5 การจัดการการสมัครสมาชิก ระบบสามารถบันทึกการสมัครแพ็คเกจของสมาชิก โดยเชื่อมโยงข้อมูล สมาชิกเข้ากับแพ็คเกจที่เลือก พร้อมบันทึกวันที่เริ่มต้นใช้งานและคำนวณวันหมดอายุโดยอัตโนมัติจากจำนวนวันของแพ็คเกจ ตามสูตร

วันหมดอายุ = วันเริ่มต้นใช้งาน + จำนวนวันของแพ็คเกจ

ผู้ใช้ไม่จำเป็นต้องกำหนดวันหมดอายุเอง เนื่องจากระบบใช้ข้อมูลวันที่เริ่มต้นและระยะเวลาของแพ็คเกจในการคำนวณ

1.3.6 การจัดการนัดหมายฝึกซ้อม ระบบสามารถบันทึกการนัดหมายระหว่างสมาชิกกับเทรนเนอร์ โดยกำหนดวันที่ เวลา สมาชิกผู้จอง และเทรนเนอร์ผู้สอน การสร้างนัดหมายใหม่จะกำหนดสถานะเริ่มต้นเป็น BOOKED และกำหนดแนวทางการเปลี่ยนสถานะเป็น COMPLETED เมื่อดำเนินการเสร็จสิ้น หรือ CANCELLED เมื่อยกเลิกการนัดหมาย ระบบสามารถบันทึกการนัดหมายระหว่างสมาชิกกับเทรนเนอร์ โดยกำหนดวันที่ เวลา สมาชิกผู้จอง และเทรนเนอร์ผู้สอน การสร้างนัดหมายใหม่จะกำหนดสถานะเริ่มต้นเป็น BOOKED และสามารถเปลี่ยนสถานะเป็น COMPLETED เมื่อดำเนินการเสร็จสิ้น หรือ CANCELLED เมื่อยกเลิกการนัดหมาย โดยใช้ State Pattern ในการจัดการการเปลี่ยนสถานะผ่านคลาสที่รับผิดชอบแต่ละสถานะ พร้อมพัฒนา Service และ Endpoint สำหรับดำเนินการเปลี่ยนสถานะนัดหมายเรียบร้อยแล้ว

1.3.7 การคำนวณราคาแพ็คเกจ ระบบสามารถคำนวณราคาแพ็คเกจตามประเภทลูกค้า โดยกำหนดหลักเกณฑ์ดังนี้

ตารางที่ 1 ตารางการคำนวณราคาแพ็คเกจ

ประเภทลูกค้า	หลักเกณฑ์การคำนวณ
ลูกค้าทั่วไป (REGULAR)	ชำระราคาเต็ม
นักศึกษา (STUDENT)	ลดราคา 20% จากราคาปกติ
ลูกค้าประจำ (LOYAL)	ลดราคา 10% จากราคาปกติ

การคำนวณราคาใช้ Strategy Pattern เพื่อแยกวิธีคำนวณของลูกค้าแต่ละประเภทออกจากกัน และรองรับการเพิ่มรูปแบบการคำนวณราคาใหม่ในอนาคตได้สะดวกขึ้น

1.3.8 การให้บริการผ่าน REST API และหน้าเว็บ ระบบให้บริการข้อมูลผ่าน REST API โดยใช้รูปแบบ JSON สำหรับการรับและส่งข้อมูล และใช้ HTTP Method ตามลักษณะการดำเนินงาน ได้แก่ GET, POST, PUT และ DELETE นอกจากนี้ ยังมีหน้าเว็บที่พัฒนาด้วย Thymeleaf สำหรับแสดงรายการข้อมูลและใช้แบบฟอร์มเพิ่มหรือแก้ไขข้อมูล โดยทั้งสองช่องทางเรียกใช้ Service ชุดเดียวกัน เพื่อให้การประมวลผลตามกฎหมายธุรกิจเป็นไปในแนวทางเดียวกัน

1.3.9 การทดสอบและการจัดเตรียมสภาพแวดล้อม โครงการครอบคลุมการทดสอบระดับ Unit Test ในชั้น Service โดยใช้ JUnit 5 และ Mockito รวมถึงการจัดเตรียม Dockerfile และ Docker Compose สำหรับรันแอปพลิเคชันร่วมกับฐานข้อมูล PostgreSQL ใน Container และการเตรียมระบบสำหรับการนำขึ้นใช้งานบน Cloud

1.4 กลุ่มผู้ใช้งานระบบ

ระบบกำหนดกลุ่มผู้ใช้งานหลักไว้ 2 กลุ่ม ได้แก่

1.4.1 พนักงานหรือผู้ดูแลฟิตเนส เป็นผู้ใช้งานหลักของระบบ มีหน้าที่จัดการข้อมูลเทรนเนอร์ สมาชิก ข้อมูลส่วนตัว แพ็กเกจสมาชิก ประวัติการสมัครสมาชิก และการนัดหมายฝึกซ้อมผ่านหน้าเว็บ ระบบช่วยรวบรวมข้อมูลที่เกี่ยวข้องไว้ในที่เดียว และลดการพึ่งพาการคำนวณหรือบันทึกข้อมูลด้วยตนเอง อย่างไรก็ตาม ระบบในขอบเขตปัจจุบันยังไม่มีการตรวจสอบตัวตนและกำหนดสิทธิ์การเข้าถึง จึงยังไม่สามารถแยกสิทธิ์การทำงานระหว่างผู้ดูแล เทรนเนอร์ และสมาชิกได้อย่างปลอดภัย

1.4.2 นักพัฒนาระบบอื่นหรือผู้พัฒนาส่วนติดต่อผู้ใช้ภายนอก เป็นผู้ใช้งาน REST API ที่ต้องการอ่านหรือจัดการข้อมูลผ่านโปรแกรมหรือส่วนติดต่อผู้ใช้ที่พัฒนาขึ้นเพิ่มเติม โดยสามารถตรวจสอบรูปแบบคำขอและคำตอบของ API ผ่าน Swagger UI ซึ่งช่วยให้เข้าใจโครงสร้างข้อมูลและทดลองเรียกใช้งานแต่ละ Endpoint ได้สะดวกขึ้น

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1. ช่วยรวบรวมข้อมูลสมาชิก เทรนเนอร์ แพ็กเกจสมาชิก และการนัดหมายฝึกซ้อมให้อยู่ในระบบเดียวกัน ทำให้การค้นหาและจัดการข้อมูลเป็นระบบมากขึ้น
2. ลดความผิดพลาดจากการคำนวณวันหมดอายุสมาชิกด้วยตนเอง โดยให้ระบบคำนวณจากวันที่เริ่มต้นและจำนวนวันของแพ็กเกจ
3. ทำให้การคำนวณราคาแพ็กเกจตามประเภทลูกค้ามีหลักเกณฑ์ที่ชัดเจน และสามารถเพิ่มรูปแบบการคำนวณใหม่ได้โดยไม่ต้องแก้ไขตรรกะเดิมทั้งหมด
4. ช่วยให้สามารถตรวจสอบและจัดการสถานะการนัดหมายฝึกซ้อมได้ตามเงื่อนไขที่ระบบกำหนด โดยใช้ State Pattern เพื่อควบคุมการเปลี่ยนสถานะให้เป็นไปตามกฎหมายธุรกิจ

5. ส่งเสริมการออกแบบซอฟต์แวร์ที่แบ่งหน้าที่อย่างชัดเจน และช่วยให้การทดสอบ การบำรุงรักษา และการพัฒนาระบบเพิ่มเติมทำได้สะดวกขึ้น
6. ส่งเสริมทักษะการพัฒนาซอฟต์แวร์เป็นทีม การควบคุมเวอร์ชัน การตรวจสอบโค้ด และการเตรียมระบบสำหรับการทำงานในสภาพแวดล้อมที่ใช้ Container

บทที่ 2

แนวคิด ทฤษฎี และเทคโนโลยีที่เกี่ยวข้อง

การพัฒนาระบบจัดการสมาชิกฟิตเนสจำเป็นต้องอาศัยแนวคิดด้านการพัฒนาซอฟต์แวร์ การออกแบบระบบ การจัดการฐานข้อมูล และการทดสอบซอฟต์แวร์ เพื่อให้ส่วนประกอบต่าง ๆ ทำงานร่วมกันได้อย่างถูกต้อง โครงการนี้จึงประยุกต์ใช้สถาปัตยกรรมแบบชั้น หลักการ SOLID รูปแบบการออกแบบซอฟต์แวร์ และเทคโนโลยีที่เหมาะสมกับการพัฒนาเว็บแอปพลิเคชัน โดยมีรายละเอียดดังต่อไปนี้

2.1 แนวคิดเกี่ยวกับระบบจัดการสมาชิกฟิตเนส

ระบบจัดการสมาชิกฟิตเนสเป็นระบบสารสนเทศที่ช่วยจัดเก็บและประมวลผลข้อมูลซึ่งเกี่ยวข้องกับ การให้บริการของฟิตเนส โดยมีองค์ประกอบสำคัญ ได้แก่ ข้อมูลสมาชิก ข้อมูลเทรนเนอร์ ข้อมูลส่วนตัวของสมาชิก แพ็กเกจสมาชิก ประวัติการสมัครแพ็กเกจ และการนัดหมายฝึกซ้อม

ข้อมูลเหล่านี้มีความสัมพันธ์กัน เช่น สมาชิกอาจมีเทรนเนอร์ประจำตัว สามารถสมัครแพ็กเกจได้หลายครั้ง และสามารถจองนัดฝึกซ้อมกับเทรนเนอร์ได้หลายรายการ ขณะที่แพ็กเกจหนึ่งรูปแบบสามารถถูกนำไปใช้ในการสมัครสมาชิกได้หลายครั้ง การออกแบบระบบจึงต้องคำนึงถึงความสัมพันธ์ระหว่างข้อมูลและข้อกำหนดในการจัดเก็บ เพื่อให้ข้อมูลมีความถูกต้องและสามารถเรียกใช้ได้ตามความต้องการ

ระบบที่พัฒนาขึ้นใช้แนวทางการจัดการข้อมูลผ่านส่วนติดต่อผู้ใช้งานเว็บและ REST API โดยให้ทั้งสองช่องทางใช้ตรรกะทางธุรกิจร่วมกัน การออกแบบดังกล่าวช่วยลดการเขียนกระบวนการทำงานซ้ำ และทำให้การตรวจสอบข้อมูล การคำนวณวันหมดอายุ และการจัดการข้อมูลอยู่ภายใต้หลักเกณฑ์เดียวกัน

2.2 แนวคิดเกี่ยวกับการพัฒนาเว็บแอปพลิเคชัน

เว็บแอปพลิเคชันเป็นซอฟต์แวร์ที่ผู้ใช้งานสามารถเข้าถึงผ่านเว็บเบราว์เซอร์ โดยมีเซิร์ฟเวอร์ทำหน้าที่รับคำขอ ประมวลผลข้อมูล และส่งผลลัพธ์กลับไปยังผู้ใช้งาน สำหรับโครงการนี้ เซิร์ฟเวอร์พัฒนาด้วย Java และ Spring Boot ส่วนหน้าเว็บใช้ Thymeleaf เพื่อสร้างเอกสาร HTML จากข้อมูลที่ได้จากระบบ การพัฒนาระบบใช้รูปแบบการทำงานสองช่องทาง ได้แก่ หน้าเว็บสำหรับพนักงานหรือผู้ดูแลฟิตเนส และ REST API สำหรับโปรแกรมที่ต้องการเข้าถึงข้อมูลโดยตรง ทั้งสองช่องทางเรียกใช้ Service ชุดเดียวกัน ทำให้กฎทางธุรกิจไม่จำเป็นต้องถูกสร้างซ้ำในแต่ละช่องทาง

การแบ่งช่องทางดังกล่าวทำให้ระบบสามารถรองรับการใช้งานผ่านหน้าเว็บในปัจจุบัน และเปิดโอกาสให้พัฒนาส่วนติดต่อผู้ใช้หรือระบบภายนอกเพิ่มเติมผ่าน API ได้ในอนาคต โดยไม่จำเป็นต้องเปลี่ยนแปลงตรรกะทางธุรกิจทั้งหมด

2.3 สถาปัตยกรรมแบบชั้น (Layered Architecture)

Layered Architecture เป็นแนวทางการออกแบบซอฟต์แวร์ที่แบ่งระบบออกเป็นชั้นตามหน้าที่ แต่ละชั้นรับผิดชอบงานเฉพาะด้านและประสานงานกับส่วนอื่นผ่านขอบเขตที่กำหนดไว้ การแบ่งหน้าที่ช่วยลดการพึ่งพากันโดยตรงระหว่างส่วนประกอบ และช่วยให้สามารถพัฒนา ทดสอบ หรือปรับปรุงแต่ละส่วนได้สะดวกขึ้น

โครงการนี้ใช้สถาปัตยกรรมแบบชั้นร่วมกับรูปแบบ MVC โดยมีส่วนประกอบหลัก ได้แก่ Controller, Service, Repository และ Entity พร้อมใช้ DTO และ Mapper เพื่อจัดการรูปแบบข้อมูลระหว่างแต่ละชั้น

2.3.1 ชั้น Presentation หรือ Controller ชั้น Presentation ทำหน้าที่รับคำขอจากผู้ใช้งานหรือโปรแกรมภายนอก ตรวจสอบข้อมูลเบื้องต้น เรียกใช้ Service และส่งผลลัพธ์กลับไปยังผู้เรียกใช้งาน ในโครงการนี้มี Controller สองลักษณะ ได้แก่ REST Controller ซึ่งรับและส่งข้อมูลในรูปแบบ JSON และ Web Controller ซึ่งประสานงานกับหน้าเว็บ Thymeleaf โดย Controller ไม่ควรเรียก Repository โดยตรงหรือรับผิดชอบตรรกะทางธุรกิจที่ซับซ้อน เพราะหน้าที่ดังกล่าวถูกกำหนดให้อยู่ในชั้น Service

2.3.2 ชั้น Business หรือ Service ชั้น Service ทำหน้าที่ประมวลผลตามกฎทางธุรกิจของระบบ เช่น การตรวจสอบว่าสมาชิกและแพ็คเกจมีอยู่จริงก่อนบันทึกการสมัคร การคำนวณวันหมดอายุของสมาชิก การกำหนดสถานะเริ่มต้นของนัดหมาย และการจัดการข้อผิดพลาดเมื่อไม่พบข้อมูล โครงการใช้ Service Interface และคลาส ServiceImpl เพื่อแยกข้อกำหนดการทำงานออกจากรายละเอียดการประมวลผลจริง Controller จึงสามารถเรียกใช้ความสามารถของ Service ผ่าน Interface ได้โดยไม่ต้องผูกกับคลาสที่ทำงานจริงโดยตรง

2.3.3 ชั้น Data Access หรือ Repository ชั้น Repository ทำหน้าที่ติดต่อกับฐานข้อมูลเพื่อเพิ่ม อ่าน แก้ไข และลบข้อมูล โดยใช้ Spring Data JPA ร่วมกับ Hibernate Repository ในโครงการนี้พัฒนาจาก Interface ที่สืบทอดจาก JpaRepository ทำให้สามารถใช้งานดำเนินงานพื้นฐาน เช่น save, findById, findAll และ deleteById ได้โดยไม่ต้องเขียนคำสั่ง SQL สำหรับการดำเนินงานทั่วไปด้วยตนเอง

2.3.4 ชั้น Domain หรือ Entity ชั้น Domain ประกอบด้วย Entity ซึ่งเป็นคลาส Java ที่ใช้แทนข้อมูลในฐานข้อมูลและกำหนดความสัมพันธ์ระหว่างข้อมูลแต่ละประเภท ตัวอย่างเช่น Member ใช้แทนข้อมูลสมาชิก Trainer ใช้แทนข้อมูลเทรนเนอร์ และ Membership ใช้แทนประวัติการสมัครแพ็คเกจ Entity มีหน้าที่แสดงโครงสร้างข้อมูลและความสัมพันธ์ที่เกี่ยวข้อง โดยรายละเอียดการประมวลผลทางธุรกิจหลักถูกจัดให้อยู่ในชั้น Service เพื่อให้โครงสร้างของระบบมีการแบ่งหน้าที่ชัดเจน

2.3.5 Data Transfer Object และ Mapper Data Transfer Object (DTO) เป็นรูปแบบข้อมูลที่ใช้สำหรับรับข้อมูลจากผู้เรียกใช้งานหรือส่งข้อมูลกลับไป โดยไม่จำเป็นต้องเปิดเผยโครงสร้าง Entity ทั้งหมด DTO ของโครงการแบ่งออกเป็น RequestDTO สำหรับรับข้อมูล และ ResponseDTO สำหรับส่งผลลัพธ์กลับ Mapper ทำหน้าที่แปลงข้อมูลระหว่าง DTO และ Entity เพื่อให้การแปลงข้อมูลอยู่ในส่วนที่รับผิดชอบ โดยเฉพาะ ตัวอย่างเช่น MembershipMapper สามารถแปลงข้อมูลจาก MembershipRequestDTO ไปเป็น Entity และแปลง Entity กลับเป็น MembershipResponseDTO ได้

การใช้ DTO และ Mapper ยังช่วยจำกัดข้อมูลที่เปิดเผยผ่าน API เช่น การกำหนดให้ MemberResponseDTO ไม่ส่งฟิลด์รหัสผ่านกลับไปใน Response

2.3.6 Dependency Injection Dependency Injection (DI) เป็นแนวทางที่ให้กรอบการทำงาน จัดเตรียมวัตถุที่คลาสต้องใช้ แทนการสร้างวัตถุของส่วนประกอบที่ต้องพึ่งพาด้วยตนเองทั้งหมด โครงการนี้ใช้ Spring ในการจัดการ Dependency และใช้ Constructor Injection โดยอาศัย @RequiredArgsConstructor ของ Lombok ร่วมกับฟิลด์ที่ประกาศเป็น private final ตัวอย่างเช่น MembershipServiceImpl รับ Repository และ Mapper ที่จำเป็นผ่าน Constructor

แนวทางนี้ช่วยลดการผูกติดระหว่างคลาส และทำให้สามารถแทนที่ Dependency ด้วยวัตถุจำลองในการทดสอบได้สะดวกขึ้น

2.4 รูปแบบ Model-View-Controller (MVC)

Model-View-Controller เป็นรูปแบบการออกแบบซอฟต์แวร์ที่แบ่งส่วนการทำงานออกเป็นสาม องค์ประกอบ ได้แก่ Model, View และ Controller เพื่อแยกข้อมูล ส่วนแสดงผล และการควบคุมการทำงาน ออกจากกัน

ในโครงการนี้ Model ประกอบด้วย Entity และ DTO ที่ใช้แทนข้อมูล View คือไฟล์ HTML ที่ประมวลผลด้วย Thymeleaf ส่วน Controller ทำหน้าที่รับคำขอ ประสานงานกับ Service และเลือกผลลัพธ์ ที่ต้องส่งกลับไปยังผู้ใช้งาน

ตัวอย่างเช่น เมื่อผู้ดูแลเว็บเปิดหน้า /trainers ระบบจะรับคำขอผ่าน Web Controller จากนั้นเรียก TrainerService เพื่อดึงข้อมูลเทรนเนอร์ แล้วนำผลลัพธ์ใส่ลงใน Model ก่อนส่งชื่อ View trainers/list กลับไปให้ Thymeleaf ประมวลผลไฟล์ templates/trainers/list.html เพื่อสร้างหน้าเว็บสำหรับแสดง รายการเทรนเนอร์

การใช้ MVC ช่วยให้ส่วนแสดงผลสามารถเปลี่ยนแปลงได้โดยไม่ต้องย้ายตรรกะการจัดการข้อมูลไปไว้ใน ไฟล์ HTML และช่วยให้ Controller มีหน้าที่ประสานงานอย่างชัดเจน

2.5 หลักการออกแบบซอฟต์แวร์ SOLID

SOLID เป็นหลักการออกแบบซอฟต์แวร์เชิงวัตถุที่ประกอบด้วยหลักการห้าประการ ซึ่งช่วยให้โครงสร้างโปรแกรมมีความชัดเจน ลดการพึ่งพาที่ไม่จำเป็น และรองรับการเปลี่ยนแปลงได้ดีขึ้น โครงงานนี้นำหลักการดังกล่าวมาประยุกต์ใช้ดังต่อไปนี้

2.5.1 Single Responsibility Principle (SRP) หลักการ Single Responsibility กำหนดให้คลาสหรือโมดูลหนึ่งมีความรับผิดชอบหลักที่ชัดเจนและมีเหตุผลในการเปลี่ยนแปลงไม่หลากหลายเกินไป ในโครงงานนี้ Controller รับผิดชอบการประสานงานกับคำขอ Service รับผิดชอบตรรกะทางธุรกิจ Repository รับผิดชอบการเข้าถึงข้อมูล และ Mapper รับผิดชอบการแปลงข้อมูล การแบ่งหน้าที่ช่วยลดการรวมความรับผิดชอบหลายประเภทไว้ในคลาสเดียว

2.5.2 Open/Closed Principle (OCP) หลักการ Open/Closed ระบุว่าองค์ประกอบซอฟต์แวร์ควรรองรับการขยายความสามารถโดยลดความจำเป็นในการแก้ไขโค้ดเดิม ตัวอย่างในโครงงานคือการใช้ Strategy Pattern สำหรับคำนวณราคา หากต้องการเพิ่มประเภทลูกค้าใหม่สามารถเพิ่มคลาสที่ Implement PricingStrategy และลงทะเบียนเป็น Spring Bean ตามรูปแบบที่กำหนด โดยไม่จำเป็นต้องเพิ่มเงื่อนไขการคำนวณทุกประเภทเข้าไปใน PricingService ทั้งหมด

2.5.3 Liskov Substitution Principle (LSP) หลักการ Liskov Substitution ระบุว่า Implementation ที่เป็นชนิดย่อยควรสามารถใช้แทนชนิดหลักได้ โดยยังคงพฤติกรรมตามสัญญาที่กำหนดไว้ ในโครงงานนี้ RegularPricing, StudentPricing และ LoyalPricing ใช้ Interface PricingStrategy ร่วมกัน จึงสามารถถูกเรียกผ่านชนิดอ้างอิงเดียวกันได้ ส่วนคลาสสถานะต่าง ๆ ถูกออกแบบให้ใช้ Interface SessionState ร่วมกันตามแนวคิด State Pattern

2.5.4 Interface Segregation Principle (ISP) หลักการ Interface Segregation เน้นการออกแบบ Interface ให้มีขอบเขตความรับผิดชอบเหมาะสม เพื่อไม่บังคับให้คลาสต้องพึ่งพาเมธอดที่ไม่จำเป็น ตัวอย่างในโครงงานคือการแยก Interface ตามความสามารถ โดย PricingStrategy รับผิดชอบการคำนวณราคา ขณะที่ SessionState กำหนดการดำเนินการเกี่ยวกับสถานะนัดหมาย นอกจากนี้ Service ของแต่ละ Entity ยังแยก Interface ตามหน้าที่ของตนเอง

2.5.5 Dependency Inversion Principle (DIP) หลักการ Dependency Inversion ระบุว่าองค์ประกอบระดับสูงไม่ควรพึ่งพารายละเอียดการทำงานระดับต่ำโดยตรง แต่ควรพึ่งพาส่งที่เป็นนามธรรม ในโครงงานนี้ Controller พึ่งพา Service Interface แทนการเรียกใช้คลาส ServiceImpl โดยตรง และ Service ใช้ Repository Interface ในการเข้าถึงข้อมูล การออกแบบเช่นนี้ช่วยให้สามารถเปลี่ยนรายละเอียดการทำงานหรือใช้ Mock Object สำหรับการทดสอบได้โดยไม่ต้องแก้ไขส่วนประกอบทั้งหมด

2.6 รูปแบบการออกแบบซอฟต์แวร์ (Design Patterns)

Design Patterns เป็นแนวทางการออกแบบที่น่ากลับมาใช้ได้สำหรับปัญหาที่พบได้บ่อยในการพัฒนาซอฟต์แวร์ โดยไม่ได้กำหนดให้ต้องใช้โค้ดรูปแบบเดียวกันทุกโครงการ แต่เป็นแนวคิดสำหรับจัดโครงสร้างคลาสและความสัมพันธ์ระหว่างวัตถุให้เหมาะสมกับปัญหา

โครงงานนี้ใช้รูปแบบการออกแบบหลายประเภท ได้แก่ Repository, Service Layer, DTO, Mapper และ Dependency Injection เพื่อจัดโครงสร้างระบบ และเลือกใช้ GoF Behavioral Patterns จำนวนสามรูปแบบ ได้แก่ Strategy, State และ Observer

2.6.1 Strategy Pattern Strategy Pattern เป็นรูปแบบการออกแบบที่แยกอัลกอริทึมหรือวิธีการทำงานที่แตกต่างกันออกเป็นคลาสหรือองค์ประกอบที่มี Interface ร่วมกัน ทำให้สามารถเลือกใช้วิธีการที่เหมาะสมตามเงื่อนไขได้โดยไม่ต้องรวมตรรกะทุกแบบไว้ในส่วนเดียว

ในระบบจัดการสมาชิกฟิตเนส Strategy Pattern ถูกนำมาใช้กับการคำนวณราคาแพ็คเกจตามประเภทลูกค้า โดยมี Interface PricingStrategy ซึ่งกำหนดเมธอด calculatePrice() และมีคลาสที่ทำหน้าที่คำนวณราคาตามประเภทลูกค้า ได้แก่ RegularPricing, StudentPricing และ LoyalPricing

RegularPricing คำนวณราคาเต็ม StudentPricing คำนวณราคาหลังหักส่วนลดร้อยละ 20 และ LoyalPricing คำนวณราคาหลังหักส่วนลดร้อยละ 10 ส่วน PricingService ทำหน้าที่เลือก Strategy ตามประเภทลูกค้าที่ได้รับ และเรียกใช้วิธีคำนวณราคาที่เกี่ยวข้อง

ระบบใช้ Map Injection ของ Spring เพื่อรวบรวม Implementation ของ PricingStrategy ตามชื่อ Bean ทำให้สามารถเลือก Strategy จากประเภทลูกค้าที่ส่งเข้ามาได้ ตัวอย่างเช่น เมื่อได้รับประเภทลูกค้า STUDENT ระบบจะเลือก StudentPricing เพื่อคำนวณราคา

2.6.2 State Pattern State Pattern เป็นรูปแบบการออกแบบที่แยกพฤติกรรมของวัตถุตามสถานะปัจจุบันออกเป็นองค์ประกอบต่าง ๆ แต่ละสถานะสามารถกำหนดได้ว่าการดำเนินการใดได้รับอนุญาตและควรเปลี่ยนไปสู่สถานะใดเมื่อมีคำสั่งเข้ามา

ในโครงงานนี้ State Pattern ถูกนำมาใช้กับสถานะการนัดหมายฝึกซ้อม โดยกำหนดสถานะ BOOKED, COMPLETED และ CANCELLED ผ่าน Interface SessionState ซึ่งมีเมธอด complete() และ cancel()

คลาส BookedState กำหนดให้การดำเนินการเสร็จสิ้นเปลี่ยนสถานะจาก BOOKED เป็น COMPLETED และการยกเลิกเปลี่ยนสถานะจาก BOOKED เป็น CANCELLED ส่วนสถานะ COMPLETED และ CANCELLED ต้องไม่อนุญาตให้ดำเนินการเปลี่ยนสถานะที่ขัดกับกฎทางธุรกิจ เช่น การยกเลิกนัดที่เสร็จสิ้นแล้ว

องค์ประกอบที่ออกแบบไว้ประกอบด้วย SessionState ซึ่งเป็น Interface, คลาส BookedState, CompletedState และ CancelledState ซึ่งเป็น Concrete State และ SessionStatusService ซึ่งทำหน้าที่ประสานงานการดึงข้อมูลนัดหมาย การเลือกสถานะ การดำเนินการ และการบันทึกสถานะใหม่

2.6.3 Observer Pattern Observer Pattern เป็นรูปแบบการออกแบบที่ใช้แจ้งเหตุการณ์ไปยังองค์ประกอบที่ลงทะเบียนรับการแจ้งเตือน เมื่อเกิดการเปลี่ยนแปลงในองค์ประกอบต้นทาง โดยช่วยลดการพึ่งพากันโดยตรงระหว่างผู้แจ้งเหตุการณ์กับผู้รับการแจ้งเตือน ทำให้สามารถเพิ่มหรือปรับเปลี่ยนผู้รับการแจ้งเตือนได้สะดวกขึ้น

ในระบบจัดการสมาชิกฟิตเนส ได้ประยุกต์ใช้ Observer Pattern สำหรับแจ้งเตือนเมื่อสถานะการนัดหมายฝึกซ้อมเปลี่ยนแปลง โดยมีส่วนที่เกี่ยวข้อง ได้แก่ SessionStatusService ซึ่งดำเนินการเปลี่ยนสถานะนัดหมาย และ Observer สำหรับรับการแจ้งเตือน ได้แก่ TrainerNotifier และ MemberNotifier ซึ่งทำหน้าที่แจ้งเหตุการณ์ที่เกี่ยวข้องกับเทรนเนอร์และสมาชิกตามลำดับ

จากผลการทดสอบ เมื่อเปลี่ยนสถานะการนัดหมายจาก BOOKED เป็น COMPLETED ผ่าน Endpoint ระบบสามารถเปลี่ยนสถานะได้สำเร็จ และ Observer ทั้งสองได้รับการแจ้งเตือน โดยแสดงผลผ่าน Console ของแอปพลิเคชัน การแจ้งเตือนดังกล่าวเป็นการแสดงผลในระบบ ไม่ใช่การส่งอีเมลหรือข้อความ SMS จริง

2.6.4 ความสัมพันธ์ระหว่างรูปแบบการออกแบบทั้งสาม Strategy, State และ Observer มีหน้าที่แตกต่างกันแต่สามารถทำงานร่วมกันในระบบได้ Strategy ใช้จัดการวิธีคำนวณราคาตามประเภทลูกค้า State ใช้ควบคุมการเปลี่ยนสถานะการนัดหมาย และ Observer มีแนวคิดสำหรับกระจายเหตุการณ์หลังจากสถานะเปลี่ยนแปลงสำเร็จ

การแบ่งหน้าที่เช่นนี้ช่วยให้แต่ละรูปแบบรับผิดชอบปัญหาเฉพาะด้าน และเปิดโอกาสให้เพิ่มความสามารถในอนาคตโดยลดการเปลี่ยนแปลงองค์ประกอบที่ไม่เกี่ยวข้อง

2.7 แนวคิดเกี่ยวกับฐานข้อมูลเชิงสัมพันธ์

ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) เป็นรูปแบบการจัดเก็บข้อมูลในลักษณะตาราง โดยแต่ละตารางประกอบด้วยแถวและคอลัมน์ และสามารถเชื่อมโยงข้อมูลระหว่างตารางผ่านคีย์ที่กำหนดไว้ โครงการนี้ใช้ PostgreSQL เป็นระบบจัดการฐานข้อมูล เนื่องจากข้อมูลของสมาชิก เทรนเนอร์ แพ็กเกจ และการนัดหมายมีความสัมพันธ์กันอย่างชัดเจน

2.7.1 Primary Key และ Foreign Key Primary Key เป็นคีย์ที่ใช้ระบุข้อมูลแต่ละรายการในตารางให้แตกต่างกัน ส่วน Foreign Key เป็นคีย์ที่ใช้เชื่อมโยงข้อมูลจากตารางหนึ่งไปยังอีกตารางหนึ่ง และช่วยรักษาความถูกต้องของความสัมพันธ์ระหว่างข้อมูล

ตัวอย่างเช่น ตาราง member มี member_id เป็น Primary Key ขณะที่ตาราง membership มี member_id เป็น Foreign Key เพื่อระบุว่าสมาชิกคนใดเป็นเจ้าของประวัติการสมัครแพ็กเกจนั้น นอกจากนี้ ตาราง training_session ยังใช้ Foreign Key เพื่อเชื่อมโยงสมาชิกและเทรนเนอร์ที่เกี่ยวข้องกับการนัดหมาย การกำหนด Foreign Key ช่วยป้องกันการอ้างอิงข้อมูลที่ไม่มีอยู่ในตารางต้นทาง และทำให้ระบบต้องคำนึงถึงลำดับการเพิ่มหรือลบข้อมูลที่มีความสัมพันธ์กัน

2.7.2 ความสัมพันธ์ระหว่างตาราง โครงการกำหนดตารางหลักไว้ 6 ตาราง และใช้ความสัมพันธ์ระหว่างข้อมูลเพื่อรองรับการดำเนินงานของระบบ โดยมีความสัมพันธ์สำคัญดังนี้

1. Member และ MemberInfo มีความสัมพันธ์แบบหนึ่งต่อหนึ่ง (One-to-One) โดยสมาชิกหนึ่งคนมีข้อมูลส่วนตัวได้ไม่เกินหนึ่งชุด และกำหนด member_id ในตาราง member_info ให้เป็น Foreign Key ที่มีข้อกำหนด UNIQUE

2. Trainer และ Member มีความสัมพันธ์แบบหนึ่งต่อหลาย (One-to-Many) โดยเทรนเนอร์หนึ่งคนสามารถดูแลสมาชิกได้หลายคน และสมาชิกสามารถไม่มีเทรนเนอร์ประจำตัวได้

3. Member และ Membership มีความสัมพันธ์แบบหนึ่งต่อหลาย โดยสมาชิกหนึ่งคนสามารถมีประวัติการสมัครแพ็คเกจได้หลายรายการ

4. MembershipPlan และ Membership มีความสัมพันธ์แบบหนึ่งต่อหลาย โดยแพ็คเกจหนึ่งรูปแบบสามารถถูกนำไปใช้ในการสมัครได้หลายครั้ง

5. Trainer และ TrainingSession มีความสัมพันธ์แบบหนึ่งต่อหลาย โดยเทรนเนอร์หนึ่งคนสามารถมีรายการนัดฝึกซ้อมได้หลายรายการ

6. Member และ TrainingSession มีความสัมพันธ์แบบหนึ่งต่อหลาย โดยสมาชิกหนึ่งคนสามารถมีรายการนัดฝึกซ้อมได้หลายรายการ

การกำหนดความสัมพันธ์เหล่านี้ช่วยลดความจำเป็นในการจัดเก็บข้อมูลเดียวกันซ้ำในหลายตาราง และทำให้สามารถเรียกดูข้อมูลที่เกี่ยวข้องผ่านการเชื่อมโยงระหว่างตารางได้อย่างเป็นระบบ

2.8 เทคโนโลยีที่ใช้ในการพัฒนาระบบ

โครงการนี้เลือกใช้เทคโนโลยีที่รองรับการพัฒนาเว็บแอปพลิเคชันด้วยภาษา Java การจัดการฐานข้อมูลเชิงสัมพันธ์ การสร้าง REST API การทดสอบซอฟต์แวร์ และการจัดเตรียมสภาพแวดล้อมสำหรับรันระบบ โดยมีรายละเอียดดังต่อไปนี้

2.8.1 ภาษา Java Java เป็นภาษาโปรแกรมเชิงวัตถุที่ใช้พัฒนาระบบฝั่งเซิร์ฟเวอร์ของโครงการ โดยกำหนด Java 17 เป็นเวอร์ชันหลัก ภาษา Java รองรับการสร้างคลาส Interface และการกำหนดความสัมพันธ์ระหว่างองค์ประกอบของระบบ ซึ่งเหมาะกับการออกแบบตาม Layered Architecture และการประยุกต์ใช้ Design Patterns

การแยกส่วนประกอบเป็นคลาส เช่น Controller, Service, Repository และ Entity ช่วยให้สามารถจัดโครงสร้างโปรแกรมตามหน้าที่ และทำให้สมาชิกในทีมพัฒนาส่วนต่าง ๆ ตามรูปแบบเดียวกันได้

2.8.2 Spring Boot Spring Boot เป็นเฟรมเวิร์กที่ใช้พัฒนาแอปพลิเคชันบน Java โดยช่วยจัดเตรียมการตั้งค่าและส่วนประกอบที่จำเป็นสำหรับการทำงานของระบบเว็บ ลดภาระในการกำหนดค่าด้วยตนเอง และรองรับการทำงานร่วมกับ Spring MVC, Spring Data JPA และ Spring Validation ในโครงงานนี้ Spring Boot ทำหน้าที่เป็นโครงสร้างหลักของแอปพลิเคชัน ตั้งแต่การรับคำขอผ่าน Controller การจัดการ Dependency การตรวจสอบข้อมูล การติดต่อฐานข้อมูล ไปจนถึงการส่งผลลัพธ์กลับไปยังผู้ใช้งาน ข้อมูลภาพรวมโครงงานระบุ Spring Boot เวอร์ชัน 4.1.1

2.8.3 Maven และ Maven Wrapper Maven เป็นเครื่องมือสำหรับจัดการ Dependency และกระบวนการ Build ของโครงการ Java โดยใช้ไฟล์ pom.xml ระบุข้อมูลโครงการ ไลบรารีที่ต้องใช้ และการตั้งค่าการสร้างไฟล์สำหรับนำไปรัน

โครงงานใช้ Maven Wrapper ผ่านไฟล์ mvnw เพื่อช่วยให้สมาชิกเรียกใช้ Maven ตามเวอร์ชันที่โครงการกำหนดได้โดยไม่ต้องติดตั้ง Maven แยกด้วยตนเองในทุกเครื่อง คำสั่ง ./mvnw test ใช้สำหรับเรียกการทดสอบอัตโนมัติ และสามารถใช้คำสั่ง Maven สำหรับสร้างไฟล์ JAR เพื่อรันแอปพลิเคชันได้

2.8.4 PostgreSQL PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ที่ใช้จัดเก็บข้อมูลหลักของระบบ ได้แก่ เทรนเนอร์ สมาชิก ข้อมูลส่วนตัวสมาชิก แพ็กเกจสมาชิก ประวัติการสมัคร และการนัดหมาย ฐานข้อมูลที่ใช้ในโครงงานมีชื่อ gym_db โดยกำหนดความสัมพันธ์ผ่าน Primary Key และ Foreign Key เพื่อรักษาความถูกต้องของข้อมูล การจัดเก็บราคาแพ็กเกจใช้ชนิดข้อมูลที่สัมพันธ์กับ BigDecimal เพื่อหลีกเลี่ยงความคลาดเคลื่อนที่อาจเกิดจากการคำนวณตัวเลขแบบเลขทศนิยมฐานสอง เช่น double

2.8.5 Spring Data JPA และ Hibernate Spring Data JPA ช่วยให้การเข้าถึงข้อมูลทำได้ผ่าน Repository Interface ขณะที่ Hibernate ทำหน้าที่เป็น Object-Relational Mapping (ORM) ซึ่งเชื่อมโยงคลาส Entity ใน Java เข้ากับตารางในฐานข้อมูล

ในโครงงานนี้ Entity ใช้แทนโครงสร้างข้อมูล และกำหนดความสัมพันธ์ด้วย Annotation เช่น @OneToOne, @ManyToOne และ @JoinColumn ส่วน Repository ใช้ดำเนินการอ่านและเขียนข้อมูลผ่าน JPA

นอกจากนี้ ยังมีการกำหนด spring.jpa.hibernate.ddl-auto=update เพื่อให้ Hibernate ปรับโครงสร้างตารางตาม Entity ในระหว่างการทำงาน การตั้งค่าดังกล่าวสะดวกต่อการพัฒนา แต่ควรพิจารณาอย่างระมัดระวังเมื่อใช้งานจริง เนื่องจากการจัดการโครงสร้างฐานข้อมูลในสภาพแวดล้อมจริงควรมีการควบคุมและตรวจสอบการเปลี่ยนแปลงอย่างเหมาะสม

2.8.6 Lombok Lombok เป็นไลบรารีที่ช่วยลดโค้ดซ้ำซ้อนในภาษา Java โดยสามารถสร้างเมธอดหรือ Constructor ผ่าน Annotation ตัวอย่างเช่น @Getter, @Setter และ @RequiredArgsConstructor โครงงานใช้ Lombok เพื่อช่วยลดโค้ดส่วนที่ไม่ใช่ตรรกะหลัก เช่น เมธอดสำหรับอ่านและกำหนดค่า รวมถึง Constructor สำหรับ Dependency Injection ทำให้โค้ดของคลาส Service และองค์ประกอบอื่น ๆ อ่านได้กระชับขึ้น

2.8.7 Spring Validation Spring Validation ใช้ตรวจสอบความถูกต้องของข้อมูลที่ได้รับเข้ามา ก่อนนำไปประมวลผล โดยใช้ Annotation เช่น @NotBlank และ @NotNull ร่วมกับ @Valid ตัวอย่างเช่น หากข้อมูลที่จำเป็นสำหรับการสมัครสมาชิกไม่ครบ ระบบสามารถปฏิเสธคำขอด้วยสถานะ HTTP 400 Bad Request แทนที่จะดำเนินการบันทึกข้อมูลที่ไม่สมบูรณ์ลงฐานข้อมูล

2.8.8 Thymeleaf Thymeleaf เป็น Template Engine ที่ใช้สร้างหน้า HTML จากข้อมูลที่ส่งมาจากฝั่งเซิร์ฟเวอร์ ในโครงการนี้ใช้สร้างหน้าเว็บสำหรับจัดการข้อมูลเทรนเนอร์ สมาชิก และข้อมูลประเภทอื่น ๆ ระบบจัดเตรียมไฟล์ Template แยกตามส่วนงาน โดยทั่วไปประกอบด้วยหน้ารายการข้อมูลและหน้าแบบฟอร์มสำหรับเพิ่มหรือแก้ไขข้อมูล การใช้ Attribute เช่น th:each, th:text, th:href, th:action, th:object และ th:field ช่วยให้สามารถแสดงข้อมูลและเชื่อมโยงแบบฟอร์มกับ DTO ได้

2.8.9 Springdoc OpenAPI และ Swagger UI Springdoc OpenAPI ใช้สร้างเอกสารอธิบาย REST API จากโครงสร้าง Controller และ DTO ของแอปพลิเคชัน โดย Swagger UI เป็นส่วนติดต่อผู้ใช้สำหรับเปิดดูเอกสารและทดลองส่งคำขอไปยัง API ในสภาพแวดล้อมการพัฒนาที่กำหนดไว้ สามารถเข้าถึง Swagger UI ผ่าน /swagger-ui/index.html และเข้าถึงเอกสาร OpenAPI ในรูปแบบ JSON ผ่าน /v3/api-docs เครื่องมือนี้ช่วยให้นักพัฒนาตรวจสอบ Endpoint รูปแบบข้อมูลที่ต้องส่ง และผลลัพธ์ที่ได้รับได้สะดวกขึ้น รวมทั้งใช้ตรวจสอบการทำงานเบื้องต้นระหว่างพัฒนาระบบได้

2.8.10 JUnit 5 และ Mockito JUnit 5 เป็นเฟรมเวิร์กสำหรับเขียนและเรียกใช้การทดสอบอัตโนมัติใน Java ส่วน Mockito ใช้สร้างวัตถุจำลอง (Mock Object) เพื่อควบคุมผลลัพธ์ของ Dependency และตรวจสอบการเรียกใช้เมธอดที่เกี่ยวข้อง โครงการใช้เครื่องมือทั้งสองในการทดสอบชั้น Service โดยจำลอง Repository และ Mapper เพื่อให้สามารถตรวจสอบตรรกะทางธุรกิจได้โดยไม่ต้องเชื่อมต่อฐานข้อมูลจริงในทุกกรณีทดสอบ ตัวอย่างการทดสอบ ได้แก่ การตรวจสอบว่าวันหมดอายุของสมาชิกคำนวณถูกต้อง การตรวจสอบว่าสถานะนัดหมายเริ่มต้นเป็น BOOKED และการตรวจสอบว่าระบบตอบกลับข้อผิดพลาดเมื่อไม่พบสมาชิกหรือแพ็คเกจที่ระบุ

2.8.11 Docker และ Docker Compose Docker เป็นเทคโนโลยีสำหรับบรรจุแอปพลิเคชันและส่วนประกอบที่จำเป็นไว้ใน Container เพื่อให้สามารถรันในสภาพแวดล้อมที่กำหนดได้ ส่วน Docker Compose ใช้กำหนดและจัดการบริการหลายตัวที่ต้องทำงานร่วมกัน โครงการใช้ Dockerfile แบบ Multi-stage Build โดยแยกขั้นตอนสร้างแอปพลิเคชันด้วย Maven และ JDK ออกจากขั้นตอนรันด้วย JRE จากนั้นใช้ Docker Compose กำหนดบริการแอปพลิเคชันและ PostgreSQL ให้ทำงานร่วมกัน

แนวทางนี้ช่วยลดความแตกต่างของสภาพแวดล้อมระหว่างเครื่องของสมาชิก และช่วยให้การเตรียมระบบสำหรับการนำไปใช้งานในสภาพแวดล้อมอื่นเป็นระบบมากขึ้น นอกจากนี้ ระบบได้รับการนำขึ้นใช้งานบน Cloud ผ่านแพลตฟอร์ม Render แล้ว โดย Docker และ Docker Compose ใช้สำหรับจัดเตรียมสภาพแวดล้อมของแอปพลิเคชันและฐานข้อมูลให้ทำงานร่วมกัน

2.8.12 Git และ GitHub Git เป็นระบบควบคุมเวอร์ชันที่ใช้ติดตามการเปลี่ยนแปลงของไฟล์และซอร์สโค้ด ส่วน GitHub เป็นแพลตฟอร์มสำหรับจัดเก็บ Repository และสนับสนุนการทำงานร่วมกันของทีม โครงการกำหนดให้สมาชิกพัฒนาใน Branch แยกตามงาน เปิด Pull Request เพื่อให้มีการตรวจสอบโค้ดก่อนรวมงานเข้าสู่ Branch main และแบ่งงานโดยใช้โครงสร้างโปรแกรมต้นแบบเดียวกัน แนวทางดังกล่าวช่วยลดความเสี่ยงจากการแก้ไขโค้ดส่วนกลางโดยไม่ประสานงาน และทำให้สามารถตรวจสอบการเปลี่ยนแปลงย้อนหลังได้

นอกจากนี้ ทีมยังมีข้อกำหนดเกี่ยวกับการไม่เผยแพร่รหัสผ่านฐานข้อมูลใน Repository ซึ่งเป็นประเด็นสำคัญในการจัดการข้อมูลการเชื่อมต่อและการทำงานร่วมกันผ่านระบบควบคุมเวอร์ชัน

2.9 แนวคิดเกี่ยวกับ REST API

Representational State Transfer (REST) เป็นแนวทางการออกแบบบริการบนเว็บที่ใช้ทรัพยากรและการดำเนินการผ่าน HTTP เพื่อให้โปรแกรมต่าง ๆ สามารถสื่อสารกันได้อย่างเป็นระบบ โครงการนี้กำหนด REST API ภายใต้เส้นทาง /api/... โดยใช้ JSON เป็นรูปแบบข้อมูลสำหรับการรับและส่งข้อมูล และกำหนด HTTP Method ให้สัมพันธ์กับการดำเนินการที่ต้องการ

2.9.1 HTTP Method ที่ใช้ในระบบ

ตารางที่ 2 ตาราง HTTP Method ที่ใช้ในระบบ

HTTP Method	หน้าที่	ตัวอย่าง
GET	เรียกดูข้อมูล	GET /api/trainers
POST	สร้างข้อมูลใหม่	POST /api/trainers
PUT	แก้ไขข้อมูล	PUT /api/trainers/{id}
DELETE	ลบข้อมูล	DELETE /api/trainers/{id}

แต่ละ Entity ใช้รูปแบบ Endpoint มาตรฐานสำหรับการเพิ่ม ดู แก้ไข และลบข้อมูล โดยมี Endpoint พิเศษสำหรับการคำนวณราคาและการเปลี่ยนสถานะนัดหมายตามรูปแบบการออกแบบที่เลือกใช้

2.9.2 HTTP Status Code ระบบใช้ HTTP Status Code เพื่อสื่อสารผลการประมวลผลกลับไปยังผู้เรียกใช้งาน เช่น

ตารางที่ 2 ตาราง HTTP Status Code

Status Code	ความหมาย	ตัวอย่างการใช้งาน
200 OK	ดำเนินการสำเร็จ	เรียกดูหรือแก้ไขข้อมูลสำเร็จ
201 Created	สร้างข้อมูลสำเร็จ	เพิ่มสมาชิกหรือสร้างการสมัครแพ็คเกจ
204 No Content	ดำเนินการสำเร็จโดยไม่มี Response Body	ลบข้อมูลสำเร็จ
400 Bad Request	คำขอหรือข้อมูลไม่ถูกต้อง	ข้อมูลไม่ผ่าน Validation หรือประเภทลูกค้าไม่ถูกต้อง
404 Not Found	ไม่พบข้อมูลที่ร้องขอ	ไม่พบสมาชิกหรือแพ็คเกจตามรหัสที่ระบุ
409 Conflict	คำขอขัดแย้งกับสถานะหรือความสัมพันธ์ของข้อมูล	ลบสมาชิกที่ยังมีข้อมูลส่วนตัว การสมัครแพ็คเกจ หรือการนัดฝึกอยู่

การใช้ Status Code อย่างเหมาะสมช่วยให้ผู้เรียกใช้งานสามารถแยกผลการดำเนินการสำเร็จออกจากข้อผิดพลาด และช่วยให้การตรวจสอบปัญหาผ่าน Swagger UI ทำได้สะดวกขึ้น

2.9.3 การตรวจสอบข้อมูลและการจัดการข้อผิดพลาด ระบบใช้ Validation ในการตรวจสอบข้อมูลเบื้องต้น และใช้ Service ตรวจสอบเงื่อนไขทางธุรกิจ เช่น การมีอยู่ของสมาชิกหรือแพ็คเกจก่อนสร้างรายการสมัครสมาชิก เมื่อพบข้อผิดพลาด Service สามารถส่งข้อผิดพลาดในรูปแบบ `ResponseStatusException` เพื่อให้ Spring แปลงเป็น HTTP Status ที่เหมาะสม ตัวอย่างเช่น เมื่อผู้ใช้ส่งคำขอสมัครสมาชิกโดยระบุ `memberId` ที่ไม่มีอยู่ในฐานข้อมูล ระบบจะตอบกลับ 404 Not Found และไม่ควรบันทึกข้อมูลการสมัครที่ไม่สมบูรณ์ลงฐานข้อมูล

2.10 แนวคิดเกี่ยวกับการทดสอบซอฟต์แวร์

การทดสอบซอฟต์แวร์เป็นกระบวนการตรวจสอบว่าระบบหรือองค์ประกอบของระบบทำงานสอดคล้องกับข้อกำหนดที่กำหนดไว้หรือไม่ โครงการนี้ใช้ Unit Test เป็นแนวทางหลักในการตรวจสอบการทำงานของชั้น Service โดยเน้นทั้งผลลัพธ์ที่ควรได้รับและการจัดการกรณีผิดพลาด

2.10.1 Unit Test Unit Test เป็นการทดสอบองค์ประกอบของโปรแกรมในขอบเขตที่แยกออกมา โดยพยายามควบคุม Dependency ที่เกี่ยวข้องเพื่อให้สามารถตรวจสอบพฤติกรรมขององค์ประกอบเป้าหมายได้โดยตรง

ในโครงงานนี้ ServiceImpl เป็นองค์ประกอบที่นำมาทดสอบ โดยใช้ Mockito จำลอง Repository และ Dependency ที่เกี่ยวข้อง เพื่อกำหนดผลลัพธ์ของการเรียกใช้ข้อมูลให้เป็นไปตามเงื่อนไขของแต่ละกรณีทดสอบ

2.10.2 รูปแบบ Given-When-Then การเขียนกรณีทดสอบใช้แนวทาง Given-When-Then เพื่อแบ่งขั้นตอนให้ชัดเจน ได้แก่

1. Given เป็นการเตรียมข้อมูลและกำหนดเงื่อนไขก่อนทดสอบ เช่น กำหนดให้ Repository ไม่พบสมาชิกที่ระบุ
2. When เป็นการเรียกเมธอดที่ต้องการทดสอบ เช่น เรียกเมธอดสร้างรายการสมัครสมาชิก
3. Then เป็นการตรวจสอบผลลัพธ์ เช่น ตรวจสอบว่าระบบส่งข้อผิดพลาด 404 และไม่มีการเรียกบันทึกข้อมูลลง Repository

แนวทางนี้ช่วยให้แต่ละกรณีทดสอบอ่านและทำความเข้าใจได้ง่าย รวมทั้งทำให้สามารถระบุเงื่อนไขและผลลัพธ์ที่คาดหวังได้อย่างชัดเจน

2.10.3 การใช้ Mock และการตรวจสอบพฤติกรรม Mockito รองรับการกำหนดผลลัพธ์จำลองผ่าน when(...).thenReturn(...) และการตรวจสอบการเรียกเมธอดผ่าน verify(...) หรือ verify(..., never())

ตัวอย่างเช่น ในกรณีที่ไม่มีพบสมาชิก ระบบควรส่งข้อผิดพลาดตามที่กำหนดและไม่เรียก save() เพื่อบันทึกรายการสมัคร การทดสอบลักษณะนี้ช่วยตรวจสอบทั้งผลลัพธ์ที่ส่งกลับและพฤติกรรมภายในที่สำคัญของ Service

2.10.4 ขอบเขตการทดสอบของโครงงาน ขอบเขตการทดสอบของโครงงานครอบคลุม Unit Test ในชั้น Service โดยใช้ JUnit 5 และ Mockito เพื่อทดสอบตรรกะทางธุรกิจของแต่ละ ServiceImpl จากผลการทดสอบที่รายงาน มีคลาสทดสอบทั้งหมด 6 คลาส และมีกรณีทดสอบที่ผ่านรวม 59 กรณี ครอบคลุมการจัดการข้อมูลเทรดเดอร์ แพ็กเกจสมาชิก การนัดหมาย การสมัครสมาชิก ข้อมูลสมาชิก และข้อมูลส่วนตัวของสมาชิก ทั้งนี้ การทดสอบดังกล่าวเป็นการทดสอบระดับ Unit Test ในชั้น Service ไม่ใช่การยืนยันว่าระบบทุกส่วนผ่านการทดสอบครบถ้วนแล้ว

2.11 แนวคิดเกี่ยวกับ Containerization และการนำระบบไปใช้งาน

Containerization เป็นแนวทางการบรรจุแอปพลิเคชันและองค์ประกอบที่จำเป็นต่อการทำงานไว้ในหน่วยที่สามารถสร้างและเรียกใช้งานได้อย่างเป็นระบบ ช่วยลดปัญหาที่เกิดจากความแตกต่างของสภาพแวดล้อมระหว่างเครื่องพัฒนาและเครื่องที่นำระบบไปใช้งาน

ในโครงงานนี้ Dockerfile ใช้สร้าง Image ของแอปพลิเคชัน Java ขณะที่ Docker Compose ใช้กำหนดบริการแอปพลิเคชันและ PostgreSQL ให้ทำงานร่วมกัน โดยบริการแอปพลิเคชันเชื่อมต่อฐานข้อมูลผ่านชื่อบริการ db ภายในเครือข่ายของ Docker Compose แทนการใช้ localhost ซึ่งหมายถึงตัว Container เองในบริบทดังกล่าว

การจัดเก็บข้อมูลฐานข้อมูลใช้ Volume ชื่อ gym_pgdata เพื่อให้ข้อมูลยังคงอยู่แม้มีการหยุดหรือลบ Container ตามเงื่อนไขการจัดการ Volume ที่กำหนดไว้

นอกจากนี้ Dockerfile ใช้ Multi-stage Build เพื่อแยกสภาพแวดล้อมสำหรับการ Build ออกจากสภาพแวดล้อมสำหรับการรันจริง ช่วยลดส่วนประกอบที่ไม่จำเป็นใน Image ขั้นสุดท้าย และทำให้การจัดเตรียมแอปพลิเคชันมีขั้นตอนที่ชัดเจน

2.12 สรุปแนวคิดและเทคโนโลยีที่เกี่ยวข้อง

จากแนวคิดและเทคโนโลยีที่ศึกษา โครงงานนี้นำหลักการออกแบบซอฟต์แวร์มาประยุกต์ใช้กับระบบจัดการสมาชิกฟิตเนส โดยใช้ Layered Architecture และ MVC เพื่อแยกหน้าที่ของส่วนรับคำขอ ตรรกะทางธุรกิจ การเข้าถึงข้อมูล และส่วนแสดงผล ใช้ SOLID เพื่อสนับสนุนการออกแบบที่มีความรับผิดชอบชัดเจน และใช้รูปแบบการออกแบบตามลักษณะของปัญหาที่ต้องแก้ไข

Strategy Pattern ใช้แยกวิธีคำนวณราคาตามประเภทลูกค้า ขณะที่ State Pattern ถูกกำหนดให้ใช้จัดการสถานะนัดหมาย และ Observer Pattern เป็นแนวคิดสำหรับแจ้งเหตุการณ์เมื่อสถานะเปลี่ยนแปลง ทั้งสามรูปแบบมีระดับความซับซ้อนต่างกัน จึงจำเป็นต้องรายงานให้สอดคล้องกับสถานะการพัฒนาจริง

ด้านเทคโนโลยี ระบบใช้ Java และ Spring Boot สำหรับการพัฒนาเว็บแอปพลิเคชัน PostgreSQL สำหรับจัดเก็บข้อมูล Spring Data JPA และ Hibernate สำหรับเชื่อมโยงวัตถุกับฐานข้อมูล Thymeleaf สำหรับหน้าเว็บ และ Springdoc OpenAPI สำหรับเอกสาร REST API ส่วนการตรวจสอบการทำงานใช้ JUnit 5 และ Mockito ขณะที่ Docker และ Git/GitHub สนับสนุนการจัดเตรียมสภาพแวดล้อมและการทำงานร่วมกันของทีม

บทที่ 3

วิธีดำเนินงาน การพัฒนา และการออกแบบระบบ

บทนี้นำเสนอการวิเคราะห์ความต้องการของระบบจัดการสมาชิกฟิตเนส การกำหนดกระบวนการทำงาน การออกแบบสถาปัตยกรรมซอฟต์แวร์ การออกแบบฐานข้อมูล และการออกแบบส่วนติดต่อผู้ใช้ เพื่อให้เห็นความสัมพันธ์ระหว่างปัญหาที่ต้องการแก้ไข ข้อกำหนดของระบบ และแนวทางการพัฒนาที่นำไปใช้จริง

3.1 การวิเคราะห์ปัญหาและความต้องการของระบบ

การวิเคราะห์ปัญหาเริ่มต้นจากความจำเป็นในการจัดการข้อมูลหลายประเภทที่เกี่ยวข้องกับการดำเนินงานของฟิตเนส หากข้อมูลสมาชิก แพ้กเกจ และการนัดหมายถูกจัดเก็บแยกจากกัน การตรวจสอบความสัมพันธ์ของข้อมูลและการติดตามสถานะอาจทำได้ไม่สะดวก นอกจากนี้ การคำนวณวันหมดอายุด้วยตนเองและการกำหนดราคาตามประเภทลูกค้ายังมีโอกาสเกิดข้อผิดพลาดจากการดำเนินงานที่ไม่เป็นมาตรฐานเดียวกัน

จากปัญหาดังกล่าว จึงกำหนดความต้องการของระบบให้สามารถจัดการข้อมูลที่เกี่ยวข้องผ่านเว็บแอปพลิเคชันและ REST API โดยแยกการประมวลผลตามหน้าที่ และกำหนดกฎทางธุรกิจที่สำคัญไว้ในชั้น Service

3.1.1 ปัญหาและแนวทางแก้ไข

ตารางที่ 3 ตารางปัญหาและแนวทางแก้ไขการวิเคราะห์และออกแบบระบบ

ปัญหาที่พบ	แนวทางที่กำหนดในระบบ
ข้อมูลสมาชิกและแพ็คเกจการจัดกระจาย	จัดเก็บข้อมูลในฐานข้อมูล PostgreSQL และเชื่อมโยงข้อมูลผ่านคีย์
ตรวจสอบแพ็คเกจและวันหมดอายุได้ยาก	จัดเก็บประวัติการสมัครและคำนวณวันหมดอายุอัตโนมัติ
ติดตามสถานะการนัดฝึกซ้อมได้ไม่สะดวก	บันทึกสถานะนัดหมายและออกแบบแนวทางการเปลี่ยนสถานะ
คำนวณราคาตามประเภทลูกค้าไม่เป็นมาตรฐานเดียวกัน	ใช้ Strategy Pattern แยกวิธีคำนวณราคา
การจัดการข้อมูลผ่านโปรแกรมภายนอกทำได้ไม่สะดวก	จัดเตรียม REST API และเอกสาร Swagger UI
โค้ดมีโอกาสซ้ำซ้อนเมื่อแบ่งงานพัฒนา	กำหนดสถาปัตยกรรมและรูปแบบการเขียนโค้ดร่วมกัน

3.2 การกำหนดความต้องการของระบบ

ความต้องการของระบบแบ่งเป็นความต้องการเชิงหน้าที่ (Functional Requirements) และความต้องการที่ไม่ใช่เชิงหน้าที่ (Non-functional Requirements) เพื่อแยกความสามารถที่ระบบต้องให้บริการออกจากข้อกำหนดเกี่ยวกับโครงสร้างและวิธีการพัฒนา

3.2.1 ความต้องการเชิงหน้าที่ ความต้องการเชิงหน้าที่กำหนดสิ่งที่ระบบต้องสามารถดำเนินการได้ โดยมีรายละเอียดดังนี้

FR-01 การจัดการเทรนเนอร์ ระบบต้องสามารถเพิ่ม แสดงรายละเอียด แก้ไข และลบข้อมูลเทรนเนอร์ โดยจัดเก็บชื่อ เบอร์โทรศัพท์ และความเชี่ยวชาญ

FR-02 การจัดการสมาชิก ระบบต้องสามารถเพิ่ม แสดงรายละเอียด แก้ไข และลบข้อมูลสมาชิกรวมถึงจัดเก็บชื่อผู้ใช้ อีเมล รหัสผ่าน เบอร์โทรศัพท์ และข้อมูลเทรนเนอร์ประจำตัวตามโครงสร้างที่กำหนด โดยไม่ส่งรหัสผ่านกลับใน Response ของสมาชิก

FR-03 การจัดการข้อมูลส่วนตัว ระบบต้องสามารถจัดการข้อมูลส่วนตัวของสมาชิก และกำหนดความสัมพันธ์แบบหนึ่งต่อหนึ่งระหว่างสมาชิกกับข้อมูลส่วนตัว

FR-04 การจัดการแพ็คเกจสมาชิก ระบบต้องสามารถเพิ่ม แสดงรายละเอียด แก้ไข และลบแพ็คเกจสมาชิก โดยจัดเก็บชื่อแพ็คเกจ ราคา และจำนวนวันที่แพ็คเกจมีผลใช้งาน

FR-05 การจัดการการสมัครสมาชิก ระบบต้องสามารถบันทึกการสมัครแพ็คเกจของสมาชิก โดยตรวจสอบว่าสมาชิกและแพ็คเกจที่อ้างถึงมีอยู่จริง และคำนวณวันหมดอายุจากวันที่เริ่มต้นกับจำนวนวันของแพ็คเกจ

FR-06 การจัดการนัดหมายฝึกซ้อม ระบบต้องสามารถบันทึกการนัดหมายโดยเชื่อมโยงสมาชิกกับเทรนเนอร์ พร้อมระบุวันที่ เวลา และสถานะของนัดหมาย โดยสถานะเริ่มต้นเป็น BOOKED และกำหนดแนวทางเปลี่ยนสถานะเป็น COMPLETED หรือ CANCELLED

FR-07 การคำนวณราคาแพ็คเกจ ระบบต้องสามารถคำนวณราคาแพ็คเกจตามประเภทลูกค้า ได้แก่ ลูกค้าทั่วไป นักศึกษา และลูกค้าประจำ โดยใช้ส่วนลดตามหลักเกณฑ์ที่กำหนด

FR-08 การให้บริการข้อมูลผ่าน REST API ระบบต้องรองรับการเพิ่ม อ่าน แก้ไข และลบข้อมูลผ่าน REST API โดยใช้ JSON เป็นรูปแบบข้อมูลรับเข้าและส่งออก พร้อมแสดงเอกสาร API ผ่าน Swagger UI

FR-09 การจัดการข้อมูลผ่านหน้าเว็บ ระบบต้องมีหน้าเว็บสำหรับแสดงรายการข้อมูลและแบบฟอร์มเพิ่มหรือแก้ไขข้อมูลแต่ละส่วน โดยเรียกใช้ Service ร่วมกับ REST API

FR-10 การตรวจสอบข้อผิดพลาด ระบบต้องตรวจสอบข้อมูลที่จำเป็น และจัดการกรณีไม่พบข้อมูลหรือเกิดข้อขัดแย้งของข้อมูลด้วย HTTP Status Code ที่เหมาะสม เช่น 400, 404 และ 409

3.2.2 ความต้องการที่ไม่ใช่เชิงหน้าที่

NFR-01 การแบ่งส่วนประกอบของระบบ ระบบต้องแยกหน้าที่ของ Controller, Service, Repository และ Entity ให้ชัดเจน เพื่อจำกัดการพึ่งพากันระหว่างส่วนประกอบ

NFR-02 การจัดการรูปแบบข้อมูล ระบบต้องใช้ DTO และ Mapper เพื่อแยกรูปแบบข้อมูลที่รับและส่งออกจาก Entity และจำกัดข้อมูลที่ไม่ควรเปิดเผย เช่น รหัสผ่านของสมาชิก

NFR-03 การบำรุงรักษาและขยายระบบ ระบบควรออกแบบให้สามารถเพิ่มความสามารถได้โดยลดผลกระทบต่อส่วนประกอบเดิม และประยุกต์ใช้หลัก SOLID กับ Design Patterns ตามความเหมาะสม

NFR-04 การทดสอบซอฟต์แวร์ ระบบต้องมี Unit Test สำหรับตรวจสอบตรรกะของชั้น Service โดยใช้ JUnit 5 และ Mockito ตามขอบเขตที่กำหนดไว้

NFR-05 การเตรียมสภาพแวดล้อม ระบบต้องมีการเตรียม Dockerfile และ Docker Compose สำหรับรันแอปพลิเคชันร่วมกับฐานข้อมูล PostgreSQL

NFR-06 การทำงานร่วมกันของทีม การพัฒนาต้องใช้ Git และ GitHub ในการควบคุมเวอร์ชัน โดยกำหนด Branch แยกตามงานและตรวจสอบการเปลี่ยนแปลงผ่าน Pull Request ก่อนรวมเข้าสู่ Branch หลัก

3.2.3 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนาระบบ

การพัฒนาระบบ Gym Member Management System ใช้เครื่องมือและเทคโนโลยีที่สนับสนุนการพัฒนาเว็บแอปพลิเคชัน การจัดการฐานข้อมูล การทดสอบซอฟต์แวร์ และการเตรียมสภาพแวดล้อมสำหรับการทำงาน โดยมีรายละเอียดดังนี้

1. Java 17 ใช้เป็นภาษาหลักในการพัฒนาระบบและเขียนโปรแกรมสำหรับประมวลผลข้อมูลและกฎทางธุรกิจ
2. Spring Boot ใช้เป็นเฟรมเวิร์กหลักในการพัฒนาเว็บแอปพลิเคชันและ REST API รวมถึงสนับสนุนการจัดการส่วนประกอบต่าง ๆ ของระบบ
3. PostgreSQL ใช้เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์สำหรับจัดเก็บข้อมูลเทรนเนอร์สมาชิก ข้อมูลส่วนตัวของสมาชิก แผนสมาชิก การสมัครสมาชิก และตารางฝึกซ้อม
4. Spring Data JPA และ Hibernate ใช้สำหรับเชื่อมต่อและจัดการข้อมูลระหว่างแอปพลิเคชันกับฐานข้อมูล โดยช่วยให้สามารถดำเนินการกับข้อมูลผ่าน Entity และ Repository
5. Thymeleaf ใช้พัฒนาส่วนติดต่อผู้ใช้ในรูปแบบหน้าเว็บ HTML โดยรับข้อมูลจากระบบมาแสดงผลและรองรับแบบฟอร์มสำหรับจัดการข้อมูล
6. Maven ใช้จัดการการตั้งค่าโครงการ ไลบรารีที่จำเป็น และกระบวนการคอมไพล์และทดสอบโปรแกรม

7. Springdoc OpenAPI และ Swagger UI ใช้จัดทำเอกสาร REST API และสนับสนุนการตรวจสอบหรือทดลองเรียกใช้งาน API ผ่านหน้าเว็บ

8. Spring Validation ใช้ตรวจสอบความถูกต้องของข้อมูลที่รับเข้าระบบตามข้อกำหนดที่กำหนดไว้

9. JUnit 5 และ Mockito ใช้ทดสอบการทำงานระดับหน่วย โดย JUnit 5 ใช้กำหนดและเรียกใช้กรณีทดสอบ ส่วน Mockito ใช้จำลองการทำงานขององค์ประกอบที่เกี่ยวข้องเพื่อทดสอบตรรกะของระบบแยกจากฐานข้อมูลจริง

10. Docker และ Docker Compose ใช้จัดเตรียมสภาพแวดล้อมสำหรับเรียกใช้งานแอปพลิเคชันและฐานข้อมูล PostgreSQL ให้สามารถทำงานร่วมกันได้ตามการตั้งค่าที่กำหนด

11. Git และ GitHub ใช้ควบคุมเวอร์ชันซอร์สโค้ด แบ่งงานพัฒนาผ่าน Branch และสนับสนุนการตรวจสอบและรวมโค้ดของสมาชิกในทีม

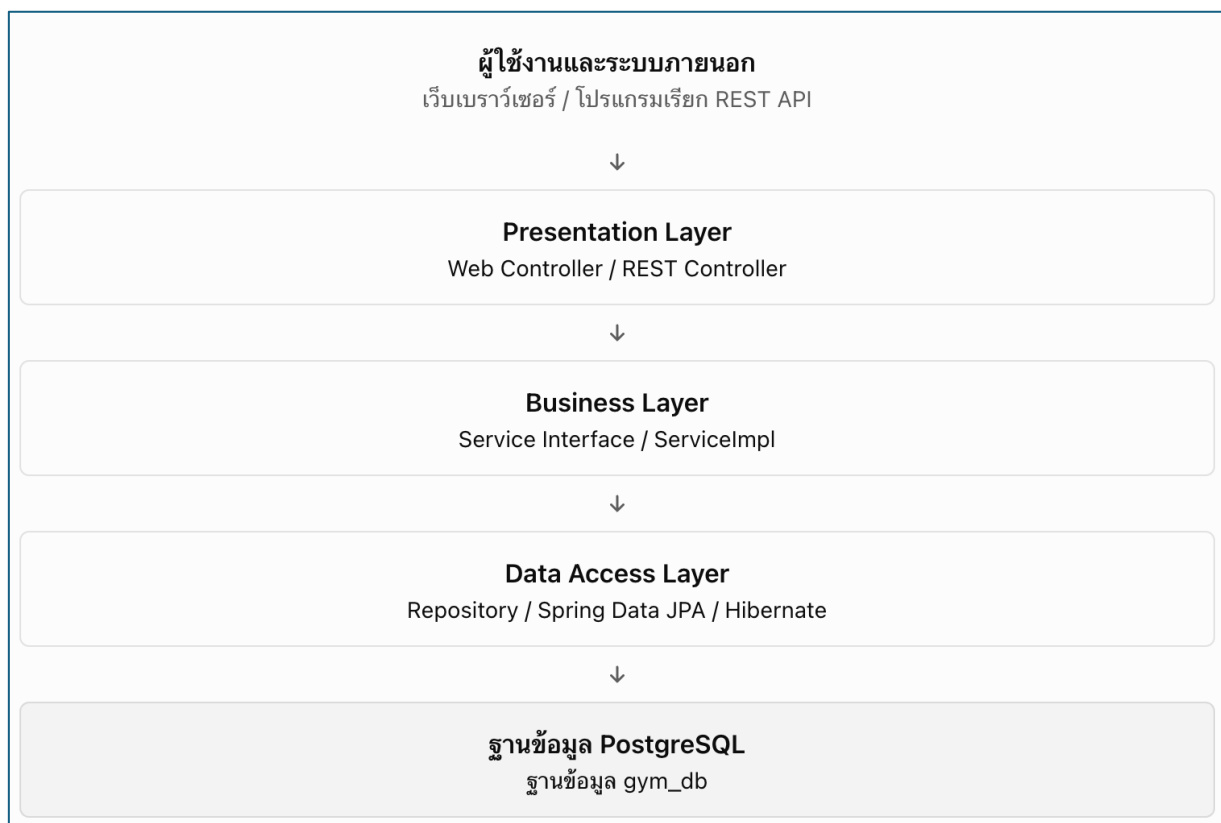
เครื่องมือและเทคโนโลยีดังกล่าวทำงานร่วมกันเพื่อสนับสนุนการพัฒนา ระบบ ตั้งแต่การรับคำขอและประมวลผลข้อมูล การจัดเก็บข้อมูลในฐานข้อมูล การแสดงผลผ่านหน้าเว็บ ไปจนถึงการทดสอบและการจัดเตรียมสภาพแวดล้อมสำหรับเรียกใช้งานระบบ

3.3 การออกแบบสถาปัตยกรรมระบบ

ระบบใช้ Layered Architecture ร่วมกับ MVC โดยแบ่งส่วนประกอบตามหน้าที่ เพื่อให้การรับคำขอ การประมวลผลทางธุรกิจ การเข้าถึงข้อมูล และการแสดงผลไม่รวมอยู่ในส่วนเดียวกัน องค์ประกอบหลักของระบบประกอบด้วย

1. Presentation Layer รับคำขอจากเว็บเบราว์เซอร์หรือ REST API และส่งต่อไปยัง Service
2. Business Layer ประมวลผลกฎทางธุรกิจและตรวจสอบเงื่อนไขของข้อมูล
3. Data Access Layer ติดต่อฐานข้อมูลผ่าน Repository
4. Domain Layer ใช้ Entity แทนข้อมูลและความสัมพันธ์ในฐานข้อมูล
5. DTO และ Mapper จัดการรูปแบบข้อมูลที่ใช้รับเข้าและส่งออกจากระบบ

3.3.1 โครงสร้างการทำงานของระบบ



ภาพที่ 1 แผนภาพสถาปัตยกรรมการทำงานของระบบที่สรุปจากโครงสร้างซอฟต์แวร์

DTO และ Mapper ทำหน้าที่สนับสนุนการแปลงข้อมูลระหว่างชั้นรับคำขอกับ Entity ส่วน Entity ใช้กำหนดข้อมูลและความสัมพันธ์ที่จัดเก็บในฐานข้อมูล

3.3.2 โครงสร้างแพ็คเกจ

โครงสร้างซอร์สโค้ดหลักอยู่ภายใต้แพ็คเกจ com.gym.membership โดยแบ่งเป็นส่วนต่าง ๆ ดังนี้

ตารางที่ 4 ตารางโครงสร้างแพ็คเกจ

แพ็คเกจหรือโฟลเดอร์	หน้าที่
controller/	REST Controller สำหรับจัดการคำขอ API
web/	Web Controller สำหรับหน้าเว็บ Thymeleaf
service/	Service Interface และ ServiceImpl
dto/	RequestDTO และ ResponseDTO
mapper/	คลาสแปลงข้อมูลระหว่าง DTO กับ Entity

domain/entity/	Entity ทั้ง 6 ประเภท
domain/repository/	Repository สำหรับเข้าถึงข้อมูล
pattern/strategy/	องค์ประกอบที่เกี่ยวข้องกับ Strategy Pattern
pattern/state/	องค์ประกอบที่เกี่ยวข้องกับ State Pattern
src/main/resources/templates/	ไฟล์ HTML สำหรับหน้าเว็บ
test/com/gym/membership/service/	Unit Test ของ Service

การจัดโครงสร้างดังกล่าวทำให้สมาชิกในทีมสามารถพัฒนาส่วนต่าง ๆ ตามขอบเขตงานได้ โดยใช้รูปแบบโค้ดและตำแหน่งจัดเก็บไฟล์ที่สอดคล้องกัน

3.4 การออกแบบฐานข้อมูล

ฐานข้อมูลของระบบมีชื่อ gym_db และใช้ PostgreSQL เป็นระบบจัดการฐานข้อมูล โดยกำหนดตารางหลัก 6 ตาราง ได้แก่ trainer, member, member_info, membership_plan, membership และ training_session

การออกแบบใช้ Primary Key เพื่อระบุข้อมูลแต่ละรายการ และใช้ Foreign Key เพื่อเชื่อมโยงข้อมูลที่เกี่ยวข้อง นอกจากนี้ ยังใช้ข้อกำหนด UNIQUE ในความสัมพันธ์แบบหนึ่งต่อหนึ่ง และกำหนดฟิลด์ที่จำเป็นให้ไม่สามารถเป็นค่าว่างได้ตามข้อกำหนดของข้อมูล

3.4.1 ตาราง trainer ตาราง trainer ใช้จัดเก็บข้อมูลเทรนเนอร์ ประกอบด้วยรหัสเทรนเนอร์ ชื่อ เบอร์โทรศัพท์ และความเชี่ยวชาญ โดย trainer_id เป็น Primary Key และใช้เชื่อมโยงกับสมาชิกและนัดหมายฝึกซ้อม

3.4.2 ตาราง member ตาราง member ใช้จัดเก็บบัญชีสมาชิก ประกอบด้วยรหัสสมาชิก ชื่อผู้ใช้ อีเมล รหัสผ่าน เบอร์โทรศัพท์ และรหัสเทรนเนอร์ประจำตัว

ฟิลด์ member_id เป็น Primary Key ส่วน trainer_id เป็น Foreign Key ที่อ้างอิงถึงตาราง trainer และสามารถเป็นค่าว่างได้เมื่อสมาชิกไม่มีเทรนเนอร์ประจำตัว

3.4.3 ตาราง member_info ตาราง member_info ใช้จัดเก็บข้อมูลส่วนตัวของสมาชิก ได้แก่ รหัสข้อมูลส่วนตัว รหัสสมาชิก ชื่อและนามสกุล เพศ วันเกิด และข้อมูลผู้ติดต่อกรณีฉุกเฉิน

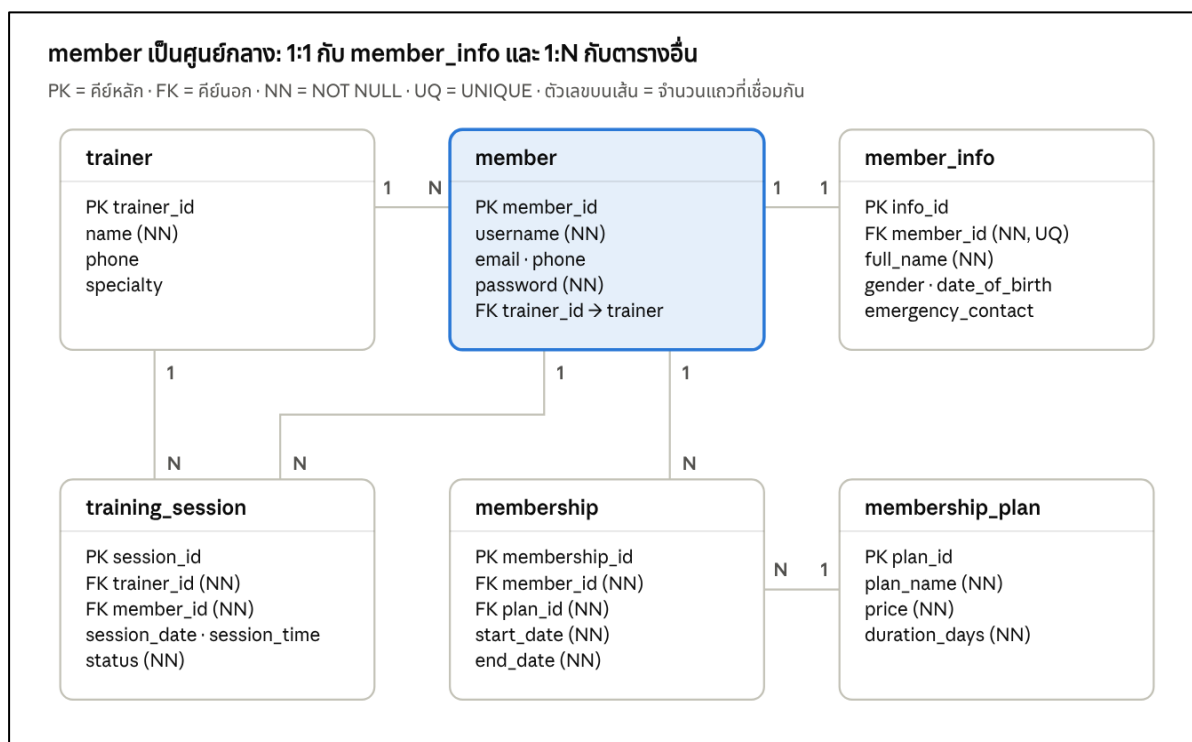
ฟิลด์ info_id เป็น Primary Key ส่วน member_id เป็น Foreign Key ที่อ้างอิงถึงตาราง member และกำหนดให้มีข้อกำหนด UNIQUE เพื่อควบคุมความสัมพันธ์แบบหนึ่งต่อหนึ่ง

3.4.4 ตาราง membership_plan ตาราง membership_plan ใช้จัดเก็บข้อมูลแพ็คเกจสมาชิก ประกอบด้วยรหัสแพ็คเกจ ชื่อแพ็คเกจ ราคา และจำนวนวันของแพ็คเกจ โดย plan_id เป็น Primary Key ฟิลด์ price ใช้ชนิดข้อมูลที่สัมพันธ์กับ BigDecimal และฐานข้อมูลเชิงสัมพันธ์ เพื่อให้เหมาะสมกับการจัดเก็บ และคำนวณค่าทางการเงิน ส่วน duration_days ใช้กำหนดจำนวนวันที่ระบบนำไปคำนวณวันหมดอายุของสมาชิก

3.4.5 ตาราง membership ตาราง membership ใช้จัดเก็บประวัติการสมัครแพ็คเกจของสมาชิก ประกอบด้วยรหัสการสมัคร รหัสสมาชิก รหัสแพ็คเกจ วันที่เริ่มต้น และวันหมดอายุ ฟิลด์ membership_id เป็น Primary Key ส่วน member_id และ plan_id เป็น Foreign Key ที่เชื่อมโยงไปยังตารางสมาชิกและแพ็คเกจตามลำดับ วันหมดอายุเป็นข้อมูลที่ระบบคำนวณจากวันที่เริ่มต้นและจำนวนวันของแพ็คเกจ

3.4.6 ตาราง training_session ตาราง training_session ใช้จัดเก็บข้อมูลนัดหมายฝึกซ้อม ประกอบด้วยรหัสนัดหมาย รหัสเทรนเนอร์ รหัสสมาชิก วันที่ เวลา และสถานะ ฟิลด์ session_id เป็น Primary Key ส่วน trainer_id และ member_id เป็น Foreign Key ที่เชื่อมโยงไปยังตารางเทรนเนอร์และสมาชิก โดยสถานะใช้ระบุว่าเป็น BOOKED, COMPLETED หรือ CANCELLED ตามกฎการทำงานที่กำหนด

3.4.7 สรุปโครงสร้างตาราง



ภาพที่ 2 แผนภาพแสดงโครงสร้างตารางข้อมูล

ความสัมพันธ์ระหว่างตารางช่วยให้ระบบสามารถติดตามข้อมูลจากสมาชิกไปยังประวัติการสมัครแพ็คเกจและนัดหมายฝึกซ้อม รวมถึงเชื่อมโยงแพ็คเกจและเทรนเนอร์ที่เกี่ยวข้องได้ โดยไม่จำเป็นต้องทำซ้ำข้อมูลหลักในทุกตาราง

3.4.8 พจนานุกรมข้อมูล (Data Dictionary)

พจนานุกรมข้อมูล (Data Dictionary) เป็นการอธิบายรายละเอียดของตารางและคอลัมน์ที่ใช้จัดเก็บข้อมูลในฐานข้อมูล gym_db ซึ่งพัฒนาด้วย PostgreSQL โดยระบบประกอบด้วยตารางหลัก 6 ตาราง ดังนี้

พจนานุกรมข้อมูลตาราง trainer ใช้จัดเก็บข้อมูลเทรนเนอร์ของฟิตเนส

ชื่อฟิลด์	ชนิดข้อมูล	คีย์และข้อกำหนด	คำอธิบาย
trainer_id	BIGINT	PK, Auto Increment	รหัสเทรนเนอร์
name	VARCHAR(255)	NOT NULL	ชื่อเทรนเนอร์
phone	VARCHAR(255)	-	เบอร์โทรศัพท์
specialty	VARCHAR(255)	-	ความเชี่ยวชาญของเทรนเนอร์

พจนานุกรมข้อมูลตาราง member ใช้จัดเก็บบัญชีและข้อมูลพื้นฐานของสมาชิก

ชื่อฟิลด์	ชนิดข้อมูล	คีย์และข้อกำหนด	คำอธิบาย
member_id	BIGINT	PK, Auto Increment	รหัสสมาชิก
username	VARCHAR(255)	NOT NULL	ชื่อผู้ใช้
email	VARCHAR(255)	-	อีเมลสมาชิก
password	VARCHAR(255)	NOT NULL	รหัสผ่าน
phone	VARCHAR(255)	-	เบอร์โทรศัพท์สมาชิก
trainer_id	BIGINT	FK → trainer, NULL ได้	รหัสเทรนเนอร์ประจำตัว

พจนานุกรมข้อมูลตาราง member_info ใช้จัดเก็บข้อมูลส่วนตัวเพิ่มเติมของสมาชิก โดยมีความสัมพันธ์แบบหนึ่งต่อหนึ่งกับตาราง member

ชื่อฟิลด์	ชนิดข้อมูล	คีย์และข้อกำหนด	คำอธิบาย
info_id	BIGINT	PK, Auto Increment	รหัสข้อมูลส่วนตัว
member_id	BIGINT	FK → member, UNIQUE, NOT NULL	รหัสสมาชิกเจ้าของข้อมูล
full_name	VARCHAR(255)	NOT NULL	ชื่อและนามสกุล
gender	VARCHAR(255)	-	เพศ
date_of_birth	DATE	-	วันเกิด
emergency_contact	VARCHAR(255)	-	ข้อมูลติดต่อกรณีฉุกเฉิน

พจนานุกรมข้อมูลตาราง membership_plan ใช้จัดเก็บรายละเอียดและราคาของแพ็คเกจสมาชิก

ชื่อฟิลด์	ชนิดข้อมูล	คีย์และข้อกำหนด	คำอธิบาย
plan_id	BIGINT	PK, Auto Increment	รหัสแพ็คเกจ
plan_name	VARCHAR(255)	NOT NULL	ชื่อแพ็คเกจ
price	NUMERIC	NOT NULL	ราคาแพ็คเกจ
duration_days	INTEGER	NOT NULL	ระยะเวลาของแพ็คเกจเป็นวัน

พจนานุกรมข้อมูลตาราง membership ใช้จัดเก็บประวัติการสมัครแพ็คเกจของสมาชิก

ชื่อฟิลด์	ชนิดข้อมูล	คีย์และข้อกำหนด	คำอธิบาย
membership_id	BIGINT	PK, Auto Increment	รหัสการสมัครสมาชิก
member_id	BIGINT	FK → member, NOT NULL	รหัสสมาชิกที่สมัคร
plan_id	BIGINT	FK → membership_plan, NOT NULL	รหัสแพ็คเกจที่เลือก
start_date	DATE	NOT NULL	วันที่เริ่มต้นสมาชิก
end_date	DATE	NOT NULL	วันหมดอายุสมาชิก

พจนานุกรมข้อมูลตาราง training_session ใช้จัดเก็บข้อมูลการนัดฝึกซ้อมระหว่างสมาชิกกับเทรนเนอร์

ชื่อฟิลด์	ชนิดข้อมูล	คีย์และข้อกำหนด	คำอธิบาย
session_id	BIGINT	PK, Auto Increment	รหัสการนัดฝึกซ้อม
trainer_id	BIGINT	FK → trainer, NOT NULL	รหัสเทรนเนอร์ผู้สอน
member_id	BIGINT	FK → member, NOT NULL	รหัสสมาชิกผู้จอง
session_date	DATE	NOT NULL	วันที่นัดฝึกซ้อม
session_time	TIME	NOT NULL	เวลานัดฝึกซ้อม
status	VARCHAR(255)	NOT NULL	สถานะการนัดฝึกซ้อม

3.5 การออกแบบกฎทางธุรกิจ

กฎทางธุรกิจเป็นเงื่อนไขที่ระบบต้องปฏิบัติตามในการประมวลผลข้อมูล เพื่อให้ผลลัพธ์เป็นไปตามข้อกำหนดของโครงการ โดยมีรายละเอียดสำคัญดังนี้

3.5.1 การคำนวณวันหมดอายุของสมาชิก เมื่อมีการสมัครแพ็คเกจ ระบบรับรหัสสมาชิก รหัสแพ็คเกจ และวันที่เริ่มต้น จากนั้นตรวจสอบว่าสมาชิกและแพ็คเกจที่ระบุมีอยู่จริง หากข้อมูลครบถ้วน ระบบจึงคำนวณวันหมดอายุจากจำนวนวันของแพ็คเกจและบันทึกข้อมูล ขั้นตอนการทำงานสามารถสรุปได้ดังนี้

1. รับข้อมูลการสมัครสมาชิกจาก RequestDTO
2. ตรวจสอบข้อมูลที่จำเป็นด้วย Validation
3. ค้นหาสมาชิกจากรหัสสมาชิก
4. ค้นหาแพ็คเกจจากรหัสแพ็คเกจ
5. หากไม่พบข้อมูลที่เกี่ยวข้อง ให้ตอบกลับ 404 Not Found
6. คำนวณวันหมดอายุจากวันที่เริ่มต้นและจำนวนวันของแพ็คเกจ
7. แปลงข้อมูลผ่าน Mapper และบันทึกผ่าน Repository
8. ส่งข้อมูลการสมัครที่บันทึกสำเร็จกลับไปยังผู้เรียกใช้งานด้วยสถานะ 201 Created

3.5.2 การคำนวณราคาตามประเภทลูกค้า ระบบใช้ Strategy Pattern เพื่อเลือกวิธีคำนวณราคาตามประเภทลูกค้าที่ระบุ โดยมีหลักเกณฑ์ดังนี้

ตารางที่ 5 ตารางการคำนวณราคาตามประเภทลูกค้า

ประเภทลูกค้า	สูตรคำนวณ	ตัวอย่างราคาเต็ม 1,000 บาท
REGULAR	ราคา × 1.00	1,000 บาท
STUDENT	ราคา × 0.80	800 บาท
LOYAL	ราคา × 0.90	900 บาท

ระบบรับรหัสแพ็คเกจและประเภทลูกค้าผ่าน Endpoint สำหรับคำนวณราคา หากไม่พบแพ็คเกจให้ตอบกลับ 404 Not Found และหากไม่รู้จักประเภทลูกค้าให้ตอบกลับ 400 Bad Request

3.5.3 การเปลี่ยนสถานะนัดหมาย ระบบกำหนดสถานะเริ่มต้นของนัดหมายเป็น BOOKED โดยออกแบบให้สถานะดังกล่าวสามารถเปลี่ยนไปเป็น COMPLETED หรือ CANCELLED ได้ตามคำสั่งที่ได้รับ การเปลี่ยนสถานะที่ไม่สอดคล้องกับกฎทางธุรกิจ เช่น การยกเลิกนัดหมายที่เสร็จสิ้นแล้ว ควรถูกปฏิเสธ

3.5.4 การป้องกันการลบสมาชิกที่มีข้อมูลส่วนตัว เมื่อมีคำขอลบสมาชิก ระบบต้องตรวจสอบว่าสมาชิกมีข้อมูลส่วนตัว การสมัครแพ็คเกจ หรือการนัดฝึกที่เชื่อมโยงอยู่หรือไม่ หากยังมีข้อมูลดังกล่าว ระบบจะปฏิเสธการลบและตอบกลับ 409 Conflict เพื่อหลีกเลี่ยงการดำเนินการที่ขัดแย้งกับความสัมพันธ์ของข้อมูล

กฎนี้ช่วยให้การจัดการข้อมูลเป็นไปตามข้อกำหนดที่กำหนดไว้ในชั้น Service และหลีกเลี่ยงการปล่อยให้ข้อผิดพลาดจากฐานข้อมูลเป็นกลไกหลักในการจัดการกรณีดังกล่าว

3.6 การออกแบบ REST API REST API เป็นช่องทางสำหรับให้ระบบภายนอกเข้าถึงข้อมูล โดยกำหนดเส้นทางตามทรัพยากรและใช้ HTTP Method ตามประเภทของการดำเนินการ

3.6.1 Endpoint สำหรับจัดการข้อมูลหลัก

ตารางที่ 6 ตาราง Endpoint สำหรับจัดการข้อมูลหลัก

ทรัพยากร	Base Path
Trainer	/api/trainers
Member	/api/members

MemberInfo	/api/member-infos
MembershipPlan	/api/membership-plans
Membership	/api/memberships
TrainingSession	/api/training-sessions

แต่ละทรัพยากรใช้ Endpoint มาตรฐานสำหรับการเรียกดูข้อมูลทั้งหมด การเรียกดูข้อมูลตามรหัส การเพิ่มข้อมูล การแก้ไขข้อมูล และการลบข้อมูล รวมถึงมีการตรวจสอบเงื่อนไขเฉพาะของแต่ละทรัพยากร

3.6.2 Endpoint สำหรับการทำงานเพิ่มเติม

นอกจาก Endpoint มาตรฐาน ระบบยังมี Endpoint สำหรับการคำนวณราคาและออกแบบการเปลี่ยนสถานะนัดหมาย ดังนี้

ตารางที่ 7 ตาราง Endpoint สำหรับการทำงานเพิ่มเติม

Method	Endpoint	หน้าที่	สถานะ
GET	/api/pricing?planId=1&customerType=STUDENT	คำนวณราคาตามประเภทลูกค้า	เสร็จแล้ว
POST	/api/training-sessions/{id}/complete	เปลี่ยนสถานะเป็น COMPLETED	เสร็จแล้ว
POST	/api/training-sessions/{id}/cancel	เปลี่ยนสถานะเป็น CANCELLED	เสร็จแล้ว

3.6.3 การออกแบบ RequestDTO และ ResponseDTO RequestDTO ใช้กำหนดรูปแบบข้อมูลที่ถูกเรียกใช้งานต้องส่งเข้ามา โดยสามารถกำหนดข้อบังคับของข้อมูลผ่าน Validation Annotation ส่วน ResponseDTO ใช้กำหนดข้อมูลที่ระบบส่งกลับหลังจากประมวลผลเสร็จ

ตัวอย่างการสมัครแพ็คเกจสมาชิก ใช้ข้อมูล memberId, planId และ startDate เป็นข้อมูลรับเข้า ส่วนผลลัพธ์ประกอบด้วยรหัสการสมัคร รหัสสมาชิก รหัสแพ็คเกจ วันที่เริ่มต้น และวันหมดอายุที่ระบบคำนวณให้การแยก DTO ช่วยให้ระบบควบคุมรูปแบบข้อมูลที่เปิดเผย และลดการส่งรายละเอียดของ Entity ที่ไม่จำเป็นไปยังผู้เรียกใช้งาน

3.7 การออกแบบส่วนติดต่อผู้ใช้

ส่วนติดต่อผู้ใช้ของระบบพัฒนาด้วย Thymeleaf โดยใช้การประมวลผล HTML ฝั่งเซิร์ฟเวอร์ (Server-Side Rendering) หน้าเว็บถูกจัดแยกตามประเภทข้อมูล และใช้ Web Controller ติดต่อกับ Service เพื่อรับข้อมูลมาแสดงผล

3.7.1 โครงสร้างหน้าเว็บ ระบบกำหนดหน้าเว็บสำหรับข้อมูลหลักทั้ง 6 ส่วน โดยแต่ละส่วนมีหน้าแสดงรายการข้อมูลและหน้าแบบฟอร์มสำหรับเพิ่มหรือแก้ไขข้อมูล ตามโครงสร้างไฟล์ดังนี้

ตารางที่ 8 ตาราง โครงสร้างหน้าเว็บ

ส่วนงาน	ไฟล์หน้าเว็บ
Trainer	trainers/list.html, trainers/form.html
Member	members/list.html, members/form.html
MemberInfo	member-infos/list.html, member-infos/form.html
MembershipPlan	membership-plans/list.html, membership-plans/form.html
Membership	memberships/list.html, memberships/form.html
TrainingSession	training-sessions/list.html, training-sessions/form.html

3.7.2 กระบวนการแสดงรายการข้อมูล เมื่อผู้ใช้งานเปิดหน้ารายการ เช่น /trainers Web Controller จะเรียก Service เพื่อดึงข้อมูลเทรนเนอร์ทั้งหมด จากนั้นเพิ่มข้อมูลลงใน Model และคืนชื่อ View ให้ Thymeleaf ประมวลผล HTML ก่อนส่งกลับไปยังเว็บเบราว์เซอร์

3.7.3 กระบวนการเพิ่มและแก้ไขข้อมูล เมื่อผู้ใช้งานเปิดแบบฟอร์มเพิ่มข้อมูล ระบบจะแสดงฟอร์มเปล่า ส่วนการแก้ไขข้อมูลจะดึงข้อมูลเดิมมาแสดงในฟอร์ม เมื่อส่งข้อมูลกลับมา Controller จะประสานงานกับ Service เพื่อประมวลผลและบันทึกข้อมูล จากนั้นเปลี่ยนเส้นทางกลับไปยังหน้ารายการ
 โครงงานใช้แนวทาง Post/Redirect/Get หลังจากส่งฟอร์ม เพื่อช่วยลดโอกาสที่การรีเฟรชหน้าเว็บจะทำให้ส่งคำขอเพิ่มข้อมูลเดิมซ้ำอีกครั้ง

ระบบมีหน้า Home และหน้า Home และปุ่มกลับหน้าหลักในทุกหน้าสำหรับเชื่อมโยงไปยังหน้าจัดการข้อมูลแต่ละประเภท ช่วยให้ผู้ใช้ใช้งานเข้าถึงฟังก์ชันต่าง ๆ ของระบบได้สะดวกขึ้น โดยหน้าเว็บสำหรับจัดการข้อมูลทำงานร่วมกับ Controller และ Service ตามโครงสร้างที่กำหนด

3.8 การออกแบบการทดสอบระบบ

การทดสอบระบบถูกกำหนดให้ตรวจสอบตรรกะของชั้น Service โดยใช้ JUnit 5 และ Mockito เพื่อทดสอบพฤติกรรมของแต่ละ ServiceImpl แยกจากฐานข้อมูลจริง

กรณีทดสอบครอบคลุมทั้งกรณีที่สามารถสำเร็จและกรณีที่เกิดข้อผิดพลาด เช่น การค้นหาข้อมูลไม่พบ การตรวจสอบความสัมพันธ์ของสมาชิกกับแพ็คเกจ และการคำนวณวันหมดอายุ

ตัวอย่างเช่น การทดสอบการสร้างรายการสมัครสมาชิกเมื่อไม่พบสมาชิก จะกำหนดให้ MemberRepository คืนค่า Optional.empty() จากนั้นเรียกเมธอดสร้างการสมัครและตรวจสอบว่าระบบส่งข้อผิดพลาด 404 Not Found พร้อมทั้งไม่เรียกบันทึกข้อมูลผ่าน MembershipRepository

การออกแบบการทดสอบลักษณะนี้ช่วยตรวจสอบว่ากฎทางธุรกิจที่กำหนดไว้ได้รับการดำเนินการตามที่คาดหวัง

3.9 การออกแบบการทำงานร่วมกันของทีม

ทีมงานใช้แนวทางแบ่งงานตาม Entity และส่วนประกอบของระบบ โดยกำหนดให้มีโครงสร้างต้นแบบที่ใช้ร่วมกัน เพื่อให้สมาชิกสามารถพัฒนาโค้ดตามรูปแบบเดียวกันและลดความแตกต่างของโครงสร้างระหว่างส่วนงาน

การควบคุมเวอร์ชันดำเนินการผ่าน Git และ GitHub โดยกำหนด Branch แยกตามงาน สมาชิกต้องส่ง Pull Request เพื่อให้มีการตรวจสอบก่อนรวมโค้ดเข้าสู่ Branch main นอกจากนี้ ยังมีการกำหนดแนวทางเกี่ยวกับการตั้งชื่อ Branch การแบ่ง Commit และการหลีกเลี่ยงการแก้ไขไฟล์ส่วนกลางโดยไม่ประสานงาน

3.10 สรุปผลการวิเคราะห์และออกแบบระบบ

จากการวิเคราะห์ความต้องการ ระบบจัดการสมาชิกฟิตเนสได้รับการออกแบบให้ครอบคลุมการจัดการข้อมูลหลัก 6 ส่วน พร้อมความสามารถในการคำนวณวันหมดอายุ การคำนวณราคา และการจัดการนัดหมายตามขอบเขตที่กำหนด

การออกแบบสถาปัตยกรรมใช้ Layered Architecture และ MVC เพื่อแยกหน้าที่ของส่วนรับคำขอ การประมวลผลทางธุรกิจ การเข้าถึงข้อมูล และการแสดงผล ส่วนฐานข้อมูลใช้ PostgreSQL พร้อมกำหนดความสัมพันธ์ระหว่างตารางให้รองรับการดำเนินงานของระบบ

นอกจากนี้ ระบบใช้ REST API เป็นช่องทางสำหรับโปรแกรมภายนอก และใช้ Thymeleaf สำหรับหน้าเว็บ โดยทั้งสองช่องทางใช้ Service ร่วมกัน การออกแบบยังนำ SOLID และ Design Patterns มาใช้เพื่อช่วยให้โครงสร้างโปรแกรมรองรับการบำรุงรักษาและการขยายความสามารถในอนาคต

บทที่ 4

การพัฒนาและผลการดำเนินงาน

บทนี้จะนำเสนอผลการพัฒนาระบบจัดการสมาชิกพิตเนสตามการออกแบบ โดยครอบคลุมโครงสร้างโปรแกรม การพัฒนาฟังก์ชันจัดการข้อมูล การเชื่อมต่อฐานข้อมูล การให้บริการ REST API การพัฒนาหน้าเว็บ การประยุกต์ใช้ Design Patterns การทดสอบซอฟต์แวร์ และการเตรียมระบบสำหรับทำงานผ่าน Docker

4.1 ภาพรวมผลการพัฒนา

ระบบได้รับการพัฒนาโดยใช้ Java และ Spring Boot เชื่อมต่อกับฐานข้อมูล PostgreSQL และจัดโครงสร้างซอฟต์แวร์ตาม Layered Architecture ร่วมกับ MVC โดยแบ่งการทำงานออกเป็น Controller, Service, Repository, Entity, DTO และ Mapper Backend CRUD ของ Entity ทั้ง 6 ส่วนและหน้าเว็บ Thymeleaf ทั้ง 6 ชุดได้รับการพัฒนาและรวมเข้าสู่ Branch main แล้ว นอกจากนี้ Unit Test ของ Service จำนวน 6 คลาส มีกรณีทดสอบที่ผ่านรวม 59 กรณีตามผลการทดสอบที่รายงาน โดยการทดสอบดังกล่าวมุ่งตรวจสอบตรรกะการทำงานในชั้น Service

4.2 ผลการพัฒนาระบบจัดการข้อมูล

4.2.1 การจัดการข้อมูลเทรนเนอร์ ระบบมีส่วนจัดการข้อมูลเทรนเนอร์ผ่าน REST API และหน้าเว็บ โดยรองรับการเพิ่มข้อมูล การเรียกดูข้อมูลทั้งหมด การเรียกดูข้อมูลตามรหัส การแก้ไขข้อมูล และการลบข้อมูล

ข้อมูลเทรนเนอร์ประกอบด้วยชื่อ เบอร์โทรศัพท์ และความเชี่ยวชาญ โดยข้อมูลที่บันทึกสามารถนำไปเชื่อมโยงกับสมาชิกและรายการนัดหมายฝึกซ้อมได้

การพัฒนาส่วนนี้ยังทำหน้าที่เป็นต้นแบบสำหรับโครงสร้างการทำงานของ Entity อื่น ๆ โดยใช้รูปแบบ Controller, Service, Repository, DTO และ Mapper เพื่อให้สมาชิกในทีมสามารถพัฒนาส่วนงานที่ได้รับมอบหมายตามแนวทางเดียวกัน

4.2.2 การจัดการข้อมูลสมาชิก ระบบรองรับการจัดการบัญชีสมาชิก ซึ่งประกอบด้วยชื่อผู้ใช้ อีเมล รหัสผ่าน เบอร์โทรศัพท์ และข้อมูลเทรนเนอร์ประจำตัว

ในส่วนการรับข้อมูล ระบบใช้ RequestDTO และ Validation เพื่อควบคุมรูปแบบข้อมูลที่ส่งเข้ามา ส่วนการตอบกลับใช้ ResponseDTO ซึ่งไม่ส่งฟิลด์รหัสผ่านกลับไปยังผู้เรียกใช้งาน

ระบบยังมีกฎการตรวจสอบความสัมพันธ์ของข้อมูล โดยหากมีค่าขอลบสมาชิกที่ยังมีข้อมูลส่วนตัว การสมัครแพ็คเกจ หรือการนัดฝึกอยู่ ระบบจะปฏิเสธคำขอและตอบกลับ 409 Conflict ตามข้อกำหนดการทดสอบ Unit Test ของ MemberServiceImpl ครอบคลุมการจัดการข้อมูลสมาชิกตามกระบวนการ

CRUD รวมถึงการตรวจสอบเงื่อนไขการลบสมาชิกที่ยังมีข้อมูลส่วนตัว การสมัครแพ็คเกจ หรือการนัดฝึก เชื่อมโยงอยู่ โดยผลการทดสอบที่รายงานมีกรณีทดสอบผ่าน 14 กรณี

4.2.3 การจัดการข้อมูลส่วนตัวของสมาชิก ระบบจัดเก็บข้อมูลส่วนตัวแยกจากข้อมูลบัญชีสมาชิก โดยประกอบด้วยชื่อและนามสกุล เพศ วันเกิด และข้อมูลผู้ติดต่อกรณีฉุกเฉิน

การแยกข้อมูลส่วนตัวออกจากบัญชีสมาชิกช่วยให้สามารถกำหนดความสัมพันธ์แบบหนึ่งต่อหนึ่ง และควบคุมข้อมูลแต่ละส่วนผ่าน Entity และ Service ที่แยกจากกันได้

4.2.4 การจัดการแพ็คเกจสมาชิก ระบบรองรับการจัดการแพ็คเกจสมาชิก โดยเก็บชื่อแพ็คเกจ ราคา และจำนวนวันของแพ็คเกจ ข้อมูลดังกล่าวถูกนำไปใช้ในการสร้างรายการสมัครสมาชิกและการคำนวณวันหมดอายุ

การจัดเก็บราคาใช้ BigDecimal เพื่อให้การคำนวณค่าทางการเงินมีความเหมาะสมมากกว่าการใช้ชนิดข้อมูลเลขทศนิยมแบบ double สำหรับการคำนวณราคาแพ็คเกจตามประเภทลูกค้า ระบบยังใช้ข้อมูลแพ็คเกจร่วมกับ PricingService เพื่อเลือกวิธีคำนวณตาม Strategy ที่กำหนดไว้

4.2.5 การจัดการการสมัครสมาชิก ระบบรองรับการสร้างและจัดการประวัติการสมัครแพ็คเกจ โดยเชื่อมโยงสมาชิกเข้ากับแพ็คเกจที่เลือก และจัดเก็บวันที่เริ่มต้นกับวันหมดอายุ

เมื่อมีการสร้างรายการสมัคร Service จะตรวจสอบการมีอยู่ของสมาชิกและแพ็คเกจก่อนดำเนินการ หากไม่พบข้อมูลที่เกี่ยวข้องจะส่งข้อผิดพลาด 404 Not Found ส่วนวันหมดอายุจะถูกคำนวณจากวันที่เริ่มต้นและจำนวนวันของแพ็คเกจโดยอัตโนมัติ

การทดสอบ MembershipServiceImpl จำนวน 9 กรณีผ่านแล้วตามข้อมูลภาพรวม โดยครอบคลุมการคำนวณวันหมดอายุ การจัดการกรณีไม่พบสมาชิกหรือแพ็คเกจ และการคำนวณวันหมดอายุใหม่เมื่อแก้ไขข้อมูลการสมัคร

4.2.6 การจัดการนัดหมายฝึกซ้อม ระบบรองรับการสร้างและจัดการข้อมูลนัดหมายฝึกซ้อม โดยข้อมูลประกอบด้วยสมาชิกผู้จอง เทรนเนอร์ผู้สอน วันที่ เวลา และสถานะการนัดหมาย การสร้างนัดหมายใหม่กำหนดสถานะเริ่มต้นเป็น BOOKED และตรวจสอบว่าข้อมูลสมาชิกกับเทรนเนอร์ที่อ้างอิงถึงมีอยู่จริงก่อนบันทึกข้อมูล ส่วนการเปลี่ยนสถานะเป็น COMPLETED หรือ CANCELLED ถูกกำหนดให้ดำเนินการผ่าน State Pattern

4.3 ผลการพัฒนา REST API

ระบบให้บริการข้อมูลผ่าน REST API ภายใต้เส้นทาง /api/... โดยใช้ JSON เป็นรูปแบบข้อมูลรับเข้าและส่งออก และกำหนด HTTP Method ตามประเภทการดำเนินการ

Endpoint มาตรฐานของแต่ละ Entity รองรับการเรียกดูข้อมูลทั้งหมด การเรียกดูข้อมูลตามรหัส การเพิ่ม การแก้ไข และการลบข้อมูล นอกจากนี้ ยังมี Endpoint สำหรับคำนวณราคาแพ็คเกจตามประเภทลูกค้า

ระบบใช้ HTTP Status Code เพื่อแสดงผลการดำเนินการ เช่น 200 OK สำหรับการดำเนินการสำเร็จ 201 Created สำหรับการสร้างข้อมูล 204 No Content สำหรับการลบข้อมูลสำเร็จ 400 Bad Request สำหรับข้อมูลหรือคำขอที่ไม่ถูกต้อง 404 Not Found สำหรับข้อมูลที่ไม่มีอยู่ และ 409 Conflict สำหรับกรณีที่คำขอขัดแย้งกับความสัมพันธ์ของข้อมูล

4.3.1 เอกสาร API ด้วย Swagger UI ระบบใช้ Springdoc OpenAPI เพื่อสร้างเอกสาร REST API และเปิดให้ตรวจสอบผ่าน Swagger UI ที่เส้นทาง /swagger-ui/index.html โดยผู้พัฒนาสามารถเลือก Endpoint ทดลองส่งคำขอ และตรวจสอบ Response ที่ได้รับได้

เครื่องมือนี้ช่วยสนับสนุนการพัฒนาและตรวจสอบการทำงานของ API โดยไม่จำเป็นต้องสร้างเครื่องมือทดสอบแยกสำหรับทุก Endpoint ทั้งนี้ การเปิดใช้งานและทดสอบผ่าน Swagger UI ไม่ได้ทดแทนการทดสอบอัตโนมัติของระบบทั้งหมด

4.4 ผลการพัฒนาส่วนติดต่อผู้ใช้

หน้าเว็บพัฒนาด้วย Thymeleaf โดยจัดแยกตามข้อมูลแต่ละประเภท และใช้ Web Controller ติดต่อกับ Service เพื่อดึงข้อมูลหรือบันทึกข้อมูล

โครงสร้างหน้าเว็บที่กำหนดไว้ประกอบด้วยหน้ารายการและหน้าแบบฟอร์มสำหรับเทรนเนอร์ สมาชิก ข้อมูลส่วนตัว แพ็กเกจสมาชิก การสมัครสมาชิก และการนัดหมายฝึกซ้อม

เมื่อผู้ใช้เปิดหน้ารายการ Web Controller จะเรียก Service เพื่อดึงข้อมูล จากนั้นนำผลลัพธ์ใส่ Model และส่งให้ Thymeleaf สร้าง HTML ส่วนการเพิ่มหรือแก้ไขข้อมูลใช้แบบฟอร์มที่เชื่อมโยงกับ RequestDTO และหลังบันทึกจะเปลี่ยนเส้นทางกลับไปยังหน้ารายการ

4.5 ผลการประยุกต์ใช้ Design Patterns

4.5.1 Strategy Pattern Strategy Pattern ถูกนำมาใช้แยกวิธีคำนวณราคาแพ็กเกจตามประเภทลูกค้า โดยมี PricingStrategy เป็น Interface และมี Implementation ได้แก่ RegularPricing, StudentPricing และ LoyalPricing

ระบบเลือกวิธีคำนวณผ่าน PricingService โดยอาศัย Map Injection ของ Spring เพื่อรวบรวม Implementation ตามชื่อ Bean ช่วยลดความจำเป็นในการรวมเงื่อนไขคำนวณราคาทุกประเภทไว้ในส่วนเดียว

4.5.2 State Pattern State Pattern ถูกออกแบบให้จัดการสถานะการนัดหมายผ่าน Interface SessionState และคลาสสถานะที่เกี่ยวข้อง โดย BookedState, CompletedState, CancelledState, SessionStatusService และ Endpoint ได้พัฒนาสำเร็จ

4.5.3 Observer Pattern Observer Pattern ได้รับการพัฒนาเพื่อแจ้งเหตุการณ์เมื่อสถานะนัดหมายเปลี่ยนแปลง โดย SessionStatusService ทำหน้าที่แจ้งเหตุการณ์ไปยัง TrainerNotifier และ MemberNotifier ซึ่งเป็น Observer สำหรับการแจ้งเตือนเทรนเนอร์และสมาชิกตามลำดับ

ผลการทดสอบ Observer Pattern ทดสอบโดยสร้างนัดฝึกใหม่ (sessionId = 2, สถานะ BOOKED) แล้วเรียก POST /api/training-sessions/2/complete ผ่าน Swagger UI ระบบตอบกลับ 200 OK และสถานะนัดเปลี่ยนเป็น COMPLETED พร้อมกันนั้น observer ทั้ง 2 ตัวได้รับแจ้งเตือนอัตโนมัติ โดยแสดงผลที่ console ของแอปพลิเคชันดังนี้

ผลนี้ยืนยันว่า SessionStatusService (Subject) แจ้งเตือนไปยัง TrainerNotifier และ MemberNotifier (Concrete Observer) ได้ครบทุกตัว โดย Service ไม่ต้องรู้จักคลาสของผู้รับแต่ละตัวโดยตรง เพราะ Spring จะรวบรวมทุกคลาสที่ implement SessionObserver มาใส่ใน List<SessionObserver> ให้อัตโนมัติ

เมื่อเรียก complete เข้ากับนัดเดิม ระบบตอบกลับ 400 Bad Request และไม่มีการแจ้งเตือนเกิดขึ้น เพราะ CompletedState โยน exception ก่อนถึงขั้นตอนบันทึกและแจ้งเตือน แสดงให้เห็นว่า observer จะได้รับแจ้งเฉพาะเมื่อเปลี่ยนสถานะสำเร็จจริงเท่านั้น

ข้อดีของการออกแบบนี้คือ ถ้าต้องการเพิ่มช่องทางแจ้งเตือนใหม่ เช่น อีเมลหรือ SMS ก็แค่สร้างคลาสใหม่ที่ implement SessionObserver โดยไม่ต้องแก้ไข SessionStatusService เลย ซึ่งเป็นไปตามหลัก Open/Closed Principle

4.6 ผลการทดสอบซอฟต์แวร์

การทดสอบอัตโนมัติใช้ JUnit 5 และ Mockito โดยเน้นตรวจสอบการทำงานของ ServiceImpl แยกจากฐานข้อมูลจริง ผ่านการจำลอง Repository และตรวจสอบผลลัพธ์ที่ได้ตามเงื่อนไขของแต่ละกรณีทดสอบ ผลการทดสอบที่ระบุในข้อมูลภาพรวมมีดังนี้

ตารางที่ 9 ตาราง ผลการทดสอบซอฟต์แวร์

คลาสทดสอบ	จำนวนกรณีทดสอบ	ขอบเขตการทดสอบ	สถานะ
TrainerServiceImplTest	7	การเพิ่ม ค้นหา แสดงรายการ แก้ไข และลบข้อมูล	ผ่าน
MembershipPlanServiceImplTest	7	การจัดการข้อมูลแพ็คเกจตาม CRUD	ผ่าน
TrainingSessionServiceImplTest	8	การสร้างนัดหมาย สถานะเริ่มต้น การค้นหา แก้ไข และลบ	ผ่าน
MembershipServiceImplTest	9	การสมัครแพ็คเกจและการคำนวณวันหมดอายุ	ผ่าน
MemberServiceImplTest	16	CRUD และการตรวจสอบการลบสมาชิกที่มีข้อมูลส่วนตัว	ผ่าน
MemberInfoServiceImplTest	12	CRUD และกรณีไม่พบสมาชิก	ผ่าน
รวมกรณีที่ผ่านมาแล้ว	59	6 คลาสทดสอบ	ผ่าน

จากตารางพบว่า มีกรณีทดสอบที่ผ่านรวม 59 กรณี จาก 6 คลาสทดสอบที่ระบุ

4.7 ผลการจัดเตรียมระบบด้วย Docker

โครงงานจัดเตรียม Dockerfile และ Docker Compose เพื่อให้แอปพลิเคชัน Spring Boot ทำงานร่วมกับฐานข้อมูล PostgreSQL ภายใน Container โดยแบ่งบริการหลักออกเป็นสองส่วน ได้แก่ app สำหรับแอปพลิเคชัน และ db สำหรับฐานข้อมูล

Dockerfile ใช้แนวทาง Multi-stage Build โดยขั้นตอนแรกใช้ Maven และ JDK 17 สำหรับสร้างไฟล์ JAR ส่วนขั้นตอนที่สองใช้ JRE 17 สำหรับรันไฟล์ JAR ที่สร้างเสร็จแล้ว วิธีนี้ช่วยแยกเครื่องมือสำหรับการ Build ออกจากสภาพแวดล้อมสำหรับรันแอปพลิเคชัน

Docker Compose กำหนดการทำงานร่วมกันของบริการทั้งสอง โดยบริการฐานข้อมูลใช้ Image postgres:17 และจัดเก็บข้อมูลผ่าน Volume ชื่อ gym_pgdata ส่วนแอปพลิเคชันรอให้บริการฐานข้อมูลผ่าน Health Check ก่อนเริ่มทำงาน และเชื่อมต่อฐานข้อมูลด้วยชื่อบริการ db

คำสั่งที่กำหนดให้ใช้สำหรับสร้างและเริ่มบริการคือ

```
docker compose up --build
```

หลังจากระบบเริ่มทำงาน ผู้ใช้สามารถเปิดหน้าเว็บผ่าน <https://gym-member-management-system.onrender.com/> เพื่อเข้าถึงหน้าหลักของเว็บไซต์ หรือเปิด Swagger UI ผ่าน

<http://localhost:8080/swagger-ui/index.html> เพื่อทดสอบและตรวจสอบการทำงานของ REST API ได้ ปัจจุบันการพัฒนาส่วน Docker เสร็จสมบูรณ์แล้ว โดยสามารถใช้ Docker ในการจัดเตรียมสภาพแวดล้อม และเรียกใช้งานระบบได้

4.8 ปัญหาที่พบระหว่างการพัฒนาและแนวทางแก้ไข

จากข้อมูลภาพรวม ปัญหาที่พบส่วนใหญ่เกี่ยวข้องกับความสัมพันธ์ของโค้ด สภาพแวดล้อมการพัฒนา และกระบวนการทำงานร่วมกันของสมาชิกในทีม โดยมีรายละเอียดดังนี้

4.8.1 ความไม่สอดคล้องของชนิดข้อมูล สมาชิกบางคนกำหนดชนิดข้อมูลของรหัสหรือราคา แตกต่างกัน เช่น ใช้ Integer แทน Long หรือใช้ double สำหรับราคา ปัญหานี้ทำให้โค้ดที่เชื่อมโยงกันไม่สามารถทำงานร่วมกันได้ตามที่คาดหวัง

แนวทางแก้ไขคือกำหนดมาตรฐานชนิดข้อมูลร่วมกัน โดยใช้ Long สำหรับรหัส และใช้ BigDecimal สำหรับข้อมูลทางการเงิน เพื่อให้การพัฒนาแต่ละส่วนสอดคล้องกับ Entity และ Repository ส่วนกลาง

4.8.2 ความไม่สอดคล้องของชื่อฟิลด์ การใช้ชื่อฟิลด์ไม่ตรงกันระหว่าง Entity กับโค้ดของสมาชิก เช่น การเรียกใช้ trainerId ในลักษณะที่ไม่ตรงกับโครงสร้างของ Entity ทำให้เกิดปัญหาในการเรียกเมธอด และเชื่อมโยงข้อมูล

ทีมแก้ไขด้วยการกำหนดแนวทางอ้างอิง Entity เดิม และปรับการเรียกใช้ให้สอดคล้องกับชนิดข้อมูลและเมธอดที่มีอยู่

4.8.3 ปัญหาการยืนยันตัวตนกับ GitHub สมาชิกพบปัญหาการ Push โค้ดไม่ได้เนื่องจากข้อผิดพลาด 403 ซึ่งเกิดจากข้อมูลบัญชี GitHub ที่ถูกบันทึกไว้ไม่ตรงกับบัญชีที่ต้องการใช้งาน แนวทางแก้ไขคือจัดการ Credential ที่บันทึกไว้ใน Keychain และใช้ข้อมูลยืนยันตัวตนที่ถูกต้อง

4.8.4 ปัญหาการจัดการรหัสผ่านฐานข้อมูล การเก็บข้อมูลการเชื่อมต่อฐานข้อมูลใน application.properties ทำให้ต้องระมัดระวังไม่ให้รหัสผ่านจริงถูก Commit ขึ้น Repository ทีมจึงกำหนดแนวทางหลีกเลี่ยงการเผยแพร่รหัสผ่าน และให้ความสำคัญกับการแยกค่าการตั้งค่าระหว่างสภาพแวดล้อม

4.8.5 ปัญหาพอร์ต 8080 ถูกใช้งาน เมื่อมีแอปพลิเคชันเดิมทำงานค้างอยู่ อาจทำให้ไม่สามารถเริ่มแอปพลิเคชันใหม่บนพอร์ต 8080 ได้ ทีมแก้ไขโดยหยุดกระบวนการเดิมหรือค้นหากระบวนการที่ใช้พอร์ตดังกล่าวก่อนเริ่มระบบใหม่

4.8.6 ปัญหาการรัน Maven และการส่งข้อมูลวันที่ การรัน Maven ไม่สำเร็จอาจเกิดจากการอยู่ผิดโฟลเดอร์โครงการหรือใช้คำสั่งไม่ตรงกับระบบปฏิบัติการ ส่วนการส่งวันที่จากแบบฟอร์มอาจเกิดปัญหาเมื่อ Spring ไม่สามารถแปลงข้อความเป็น LocalDate ได้ตามรูปแบบที่ต้องการ แนวทางแก้ไขคือรันคำสั่งจากโฟลเดอร์ที่มีไฟล์ pom.xml และกำหนด @DateTimeFormat ให้ตรงกับรูปแบบวันที่หรือเวลาที่ใช้ในแบบฟอร์ม

4.8.7 ปัญหาความสัมพันธ์ของข้อมูล การเพิ่มหรือลบข้อมูลผิดพลาดอาจทำให้เกิดข้อผิดพลาดจาก Foreign Key ที่มีการกำหนดให้เพิ่มข้อมูลแม่ก่อนข้อมูลลูก และลบข้อมูลลูกก่อนข้อมูลแม่ พร้อมกำหนดกฎทางธุรกิจเพื่อจัดการกรณีที่ไม่น่าจะเกิดให้ลบสมาชิกซึ่งยังมีข้อมูลส่วนตัว การสมัครแพ็คเกจ หรือการนัดฝึกอยู่

ปัญหาเหล่านี้แสดงให้เห็นถึงความสำคัญของการกำหนดมาตรฐานร่วมกันและการใช้โครงสร้างโปรแกรมต้นแบบ เพื่อช่วยให้สมาชิกพัฒนาส่วนต่าง ๆ ของระบบได้สอดคล้องกัน

บทที่ 5

สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการพัฒนาระบบ

โครงการ Gym Member Management System หรือระบบจัดการสมาชิกฟิตเนส จัดทำขึ้นเพื่อพัฒนาระบบสำหรับจัดเก็บและจัดการข้อมูลที่เกี่ยวข้องกับการให้บริการฟิตเนสอย่างเป็นระบบ โดยครอบคลุมข้อมูลเทรนเนอร์ สมาชิก ข้อมูลเพิ่มเติมของสมาชิก แผนสมาชิก การสมัครสมาชิก และตารางฝึกซ้อม ระบบพัฒนาด้วยภาษา Java 17 และ Spring Boot ใช้ PostgreSQL เป็นฐานข้อมูล และใช้ Thymeleaf ในการสร้างส่วนติดต่อผู้ใช้ผ่านเว็บเบราว์เซอร์

การพัฒนาระบบใช้สถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture) ร่วมกับรูปแบบ Model-View-Controller (MVC) เพื่อแยกหน้าที่การทำงานของแต่ละส่วน ได้แก่ Controller, Service, Repository และ Entity ทำให้โครงสร้างโปรแกรมมีความชัดเจน และช่วยให้สามารถพัฒนา แก้ไข และทดสอบแต่ละส่วนได้สะดวกขึ้น นอกจากนี้ยังมีการพัฒนา REST API สำหรับจัดการข้อมูล และนำ Docker มาใช้เตรียมสภาพแวดล้อมสำหรับเรียกใช้งานแอปพลิเคชันร่วมกับฐานข้อมูล

ในด้านกฎทางธุรกิจ ระบบมีการตรวจสอบความสัมพันธ์ของข้อมูลก่อนดำเนินการ เช่น การตรวจสอบสมาชิกและแผนสมาชิกก่อนสร้างรายการสมัครสมาชิก รวมถึงการคำนวณวันหมดอายุจากวันเริ่มต้นและระยะเวลาของแผน นอกจากนี้ยังมีการประยุกต์ใช้ Strategy Pattern เพื่อแยกวิธีคำนวณราคาตามประเภทลูกค้า และใช้ State Pattern เพื่อรองรับการจัดการสถานะของตารางฝึกซ้อม

สำหรับการทดสอบ มีการใช้ JUnit 5 และ Mockito ในการทดสอบระดับ Unit Test โดยคลาสทดสอบ 6 คลาสที่ดำเนินการแล้วมีกรณีทดสอบผ่านรวม 59 กรณี ผลดังกล่าวแสดงให้เห็นว่าส่วนที่ผ่านการทดสอบสามารถทำงานได้ตามเกณฑ์ที่กำหนดไว้

5.2 อภิปรายผลการพัฒนา

จากการดำเนินโครงการพบว่า การแบ่งโครงสร้างโปรแกรมตามหน้าที่ช่วยให้การจัดการโค้ดเป็นระบบมากขึ้น โดย Controller รับผิดชอบการรับคำขอ Service จัดการกฎทางธุรกิจ และ Repository ทำหน้าที่ติดต่อฐานข้อมูล การแยกหน้าที่ดังกล่าวช่วยลดการรวมคำสั่งหลายประเภทไว้ในส่วนเดียว และทำให้สามารถทดสอบการทำงานของ Service โดยใช้ Mockito จำลองส่วนที่เกี่ยวข้องได้

การประยุกต์ใช้รูปแบบการออกแบบยังช่วยให้ระบบรองรับกฎทางธุรกิจได้เหมาะสมขึ้น โดย Strategy Pattern ทำให้สามารถแยกวิธีคำนวณราคาตามประเภทลูกค้าออกจากส่วนการทำงานหลัก ส่วน State Pattern ช่วยกำหนดเงื่อนไขการเปลี่ยนสถานะของตารางฝึกซ้อมให้ชัดเจน แนวทางนี้สามารถช่วยให้การเพิ่ม

หรือแก้ไขกฎในอนาคตทำได้ง่ายขึ้น แต่ประสิทธิภาพของรูปแบบการออกแบบยังขึ้นอยู่กับให้นำไปใช้ให้ครบถ้วนและเหมาะสมกับการทำงานจริง

ในด้านการทดสอบ มีการใช้ JUnit 5 และ Mockito ในการทดสอบระดับ Unit Test ของ Service จำนวน 6 คลาส โดยมีกรณีทดสอบผ่านรวม 59 กรณี ผลดังกล่าวช่วยยืนยันพฤติกรรมของส่วนที่ได้รับการทดสอบตามกรณีที่กำหนดไว้ อย่างไรก็ตาม การทดสอบระดับ Unit Test ยังไม่ครอบคลุมการทำงานร่วมกันระหว่าง Controller, Service, Repository และฐานข้อมูลทั้งหมด จึงควรเพิ่มการทดสอบ Controller และ Integration Test เพื่อประเมินการทำงานร่วมกันของส่วนประกอบต่าง ๆ

นอกจากนี้ Docker และ Docker Compose ช่วยจัดเตรียมสภาพแวดล้อมของแอปพลิเคชันและฐานข้อมูลให้ทำงานร่วมกันได้อย่างเป็นระบบ ขณะที่ระบบได้รับการนำขึ้นใช้งานบน Cloud ผ่านแพลตฟอร์ม Render แล้ว อย่างไรก็ตาม ยังควรปรับปรุงด้านความปลอดภัย ตรวจสอบการทำงานของฟังก์ชันที่สำคัญ และเพิ่มความครอบคลุมของการทดสอบก่อนนำระบบไปใช้งานในวงกว้าง

5.3 ข้อจำกัดของระบบ

แม้ระบบจะรองรับการจัดการข้อมูลหลักหลายประเภท แต่ยังมีข้อจำกัดที่ควรได้รับการปรับปรุง ดังนี้

1. ด้านความปลอดภัย แม้ว่าระบบจะได้รับการพัฒนาและนำขึ้นใช้งานบนแพลตฟอร์ม Render แล้ว แต่ระบบยังไม่มีมอดูลยืนยันตัวตนและกำหนดสิทธิ์ผู้ใช้งานด้วย Spring Security อีกทั้งการจัดเก็บรหัสผ่านในรูปแบบข้อความธรรมดายังมีความเสี่ยงด้านความปลอดภัย จึงควรปรับปรุงกลไกการยืนยันตัวตน การกำหนดสิทธิ์ และการจัดเก็บรหัสผ่านให้เหมาะสมก่อนนำไปใช้งานจริงในระดับที่ต้องการความปลอดภัยสูง

2. ด้านการจองและการจัดการสมาชิก ระบบยังต้องเพิ่มการตรวจสอบการจองช่วงเวลาฝึกซ้อมที่ซ้ำกัน และการตรวจสอบสถานะสมาชิกก่อนทำการการจอง เพื่อป้องกันการจองที่ไม่ตรงตามเงื่อนไขการให้บริการ

3. ด้านความสมบูรณ์ของฟังก์ชัน ระบบสามารถจัดการการเปลี่ยนสถานะการนัดหมายด้วย State Pattern และแจ้งเตือนเหตุการณ์ผ่าน Observer Pattern ได้ตามผลการทดสอบที่ดำเนินการแล้ว อย่างไรก็ตาม ระบบแจ้งเตือนในปัจจุบันแสดงผลผ่าน Console ของแอปพลิเคชัน จึงยังสามารถพัฒนาต่อเพื่อให้รองรับการแจ้งเตือนผ่านช่องทางอื่น เช่น อีเมลหรือ SMS ได้ในอนาคต

4. ด้านการทดสอบ แม้จะมีการทดสอบ Unit Test ในชั้น Service จำนวน 6 คลาส และมีกรณีทดสอบผ่านรวม 59 กรณีแล้ว แต่ยังคงควรเพิ่มการทดสอบ Controller และเส้นทาง API ที่สำคัญให้ครอบคลุมรวมถึงจัดทำ Integration Test เพื่อประเมินการทำงานร่วมกันระหว่าง Controller, Service, Repository และฐานข้อมูล

5. ด้านการใช้งานและการติดตั้ง หน้าแรกและระบบนำทางส่วนกลางยังควรได้รับการปรับปรุงเพื่อให้ผู้ใช้เข้าถึงฟังก์ชันต่าง ๆ ได้สะดวกยิ่งขึ้น ขณะที่ระบบได้รับการนำขึ้นใช้งานบน Cloud ผ่านแพลตฟอร์ม

Render แล้ว โดยสามารถเข้าถึงเว็บไซต์ผ่าน URL ที่กำหนดได้ อย่างไรก็ตาม ควรตรวจสอบความถูกต้องของการทำงานและการเชื่อมต่อฐานข้อมูลอย่างต่อเนื่อง

5.4 ข้อเสนอแนะในการพัฒนาต่อ

เพื่อให้ระบบมีความสมบูรณ์และเหมาะสมต่อการใช้งานมากขึ้น ควรดำเนินการปรับปรุงในหลายด้าน เริ่มจากการพัฒนาระบบยืนยันตัวตนและกำหนดสิทธิ์ผู้ใช้งาน พร้อมเปลี่ยนการจัดเก็บรหัสผ่านให้ใช้วิธีที่ปลอดภัย และตรวจสอบการตั้งค่าที่เกี่ยวข้องกับฐานข้อมูลไม่ให้เปิดเผยข้อมูลสำคัญโดยไม่จำเป็น นอกจากนี้ ควรติดตามการทำงานของระบบบน Cloud ตรวจสอบการตั้งค่าสภาพแวดล้อมและการเชื่อมต่อฐานข้อมูล รวมถึงเตรียมแนวทางสำรองและกู้คืนข้อมูล เพื่อเพิ่มความน่าเชื่อถือในการให้บริการ

สุดท้าย ในด้านการทำงาน ควรเพิ่มเงื่อนไขตรวจสอบช่วงเวลาฝึกซ้อมที่ซ้ำกัน ตรวจสอบสถานะสมาชิกก่อนทำรายการ และพัฒนาระบบแจ้งเตือนให้รองรับช่องทางเพิ่มเติม เช่น อีเมลหรือ SMS

นอกเหนือจากการแสดงผลผ่าน Console ในปัจจุบัน นอกจากนี้ ควรตรวจสอบการเปลี่ยนสถานะของตารางฝึกซ้อมให้เป็นไปตามกฎที่กำหนด และเพิ่มการจัดการข้อผิดพลาดเพื่อให้ผู้ใช้เข้าใจสาเหตุเมื่อไม่สามารถดำเนินการได้

5.5 บทสรุป

โครงการ Gym Member Management System เป็นการประยุกต์ใช้ความรู้ด้านการพัฒนาซอฟต์แวร์เพื่อสร้างระบบจัดการสมาชิกฟิตเนส โดยมีฟังก์ชันหลักสำหรับจัดการข้อมูลและกฎทางธุรกิจ พร้อมใช้สถาปัตยกรรมแบบแบ่งชั้น รูปแบบการออกแบบซอฟต์แวร์ การทดสอบอัตโนมัติ และ Docker เพื่อสนับสนุนการพัฒนา

ผลการดำเนินงานแสดงให้เห็นว่าระบบมีโครงสร้างและฟังก์ชันหลักที่สามารถนำไปพัฒนาต่อได้ แต่ยังมีข้อจำกัดด้านความปลอดภัย การทดสอบ และฟังก์ชันที่ยังไม่สมบูรณ์ การดำเนินการปรับปรุงตามข้อเสนอแนะจะช่วยเพิ่มความน่าเชื่อถือของระบบ และทำให้สามารถนำไปต่อยอดเป็นระบบจัดการฟิตเนสที่รองรับการใช้งานจริงได้ในอนาคต

เอกสารอ้างอิง

Docker, Inc. (ม.ป.ป.). Docker Compose documentation. <https://docs.docker.com/compose/>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

JUnit Team. (ม.ป.ป.). JUnit 5 user guide. <https://junit.org/junit5/docs/current/user-guide/>

Mockito Contributors. (ม.ป.ป.). Mockito documentation. <https://site.mockito.org/>

Oracle. (ม.ป.ป.). JDK 17 documentation. <https://docs.oracle.com/en/java/javase/17/>

The PostgreSQL Global Development Group. (ม.ป.ป.). PostgreSQL documentation. <https://www.postgresql.org/docs/17/>

The Spring Data Team. (ม.ป.ป.). Spring Data JPA reference documentation. <https://docs.spring.io/spring-data/jpa/reference/>

The Thymeleaf Team. (ม.ป.ป.). Thymeleaf documentation. <https://www.thymeleaf.org/documentation>

VMware, Inc., & Spring Community. (ม.ป.ป.). Spring Boot reference documentation. <https://docs.spring.io/spring-boot/reference/>